

目录

吴恩达 NLP 课程 4	3
1. 课程简介	3
2. 第一周（机器翻译 NMT）	3
2.1 seq2seq	3
2.2 对齐	5
2.3 Attention	8
2.4 本章所用数据集	11
2.5 用 Attention 训练 NMT	13
2.6 机器翻译评估（NMTE）	17
2.7 采样与解码	20
3. 第二周	25
3.1 Trasformer 和 RNN	25
3.2 Transformer 的应用	29
3.3 点乘注意（Dot-Product Attention）	32
3.4 Casual Attention	36
3.5 multi-head attention	39
3.6 transformer-decoder	43
3.7 transformer-summarizer	47
4. 第三周	50
4.1 本周主要内容	50
4.2 NLP 中的转移学习	52
4.3 ELMO、GPT、BERT、T5	58
4.4 BERT	64
Bidirectional Encoder Representations from Transformers（BERT） ..	64
4.5 BERT 目标	67
4.6 BERT 的微调	70
4.7 T5 Transformer	71
4.8 多任务训练策略	75

4.9	GLUE 基准	79
4.10	问答 QA.....	81
5.	第四周	84
5.1	对话机器人简介	84
	Dragon 模型基于 GPT-3。它基于所有先前的回合作为输入来生成交互式故事。	
	这使得使用非常长序列的任务成为可能。看看这个!	84
5.2	Transformer 复杂度	85
5.3	LSH Attention	87
5.4	可逆层的设计初衷: 更高效的存储	91
5.5	可逆残留层	92
5.6	reformer	96
6.	附录: (参考资料)	98
	This course drew from the following resources:	98
	- Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (Raffel et al, 2019).....	98
	- Reformer: The Efficient Transformer (Kitaev et al, 2020).....	98
	- Attention Is All You Need (Vaswani et al, 2017).....	98
	- Deep contextualized word representations (Peters et al, 2018)... ..	98
	- The Illustrated Transformer (Alammar, 2018).....	98
	- The Illustrated GPT-2 (Visualizing Transformer Language Models) (Alammar, 2019).....	98
	- BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (Devlin et al, 2018).....	98
	- How GPT3 Works - Visualizations and Animations (Alammar, 2020).....	98

吴恩达 NLP 课程 4

1. 课程简介

本章节将会学习较为前沿的 NLP 知识，动手搭建一个对话机器人。

2. 第一周（机器翻译 NMT）

neural machine translation

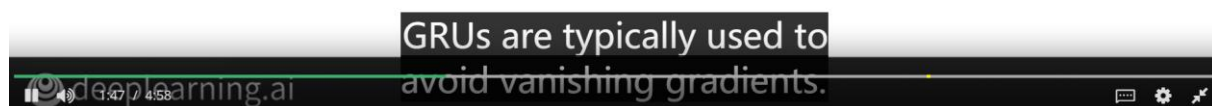
2.1 seq2seq

在神经网络搭建的机器翻译模型中，需要使用编码器和解码器。本周需要我们将英语翻译成德语，使用的是具有 LSTM 的机器翻译模型。

传统的 seq2seq 是由 Google 在 2014 年推出的，将可变长度序列映射为固定长度序列的内存中，为了避免梯度消失，使用了 LSTM、GRU：

Seq2Seq model

- Introduced by Google in 2014
- Maps variable-length sequences to fixed-length memory
- LSTMs and GRUs are typically used to overcome the vanishing gradient problem



由于 seq2seq 使用的是定长序列，因此对于较长的句子序列，往往不会有很好的效果，相反，他对较短的句子具有偏向性。因此 Seq2Seq 的强大之处在于其让输入和输出大小不同，当输入本身很大时，它就会变成它的弱点。由于编码器隐藏状态的大小是固定的，更长的输入在进入解码器的过程中将成为瓶颈。

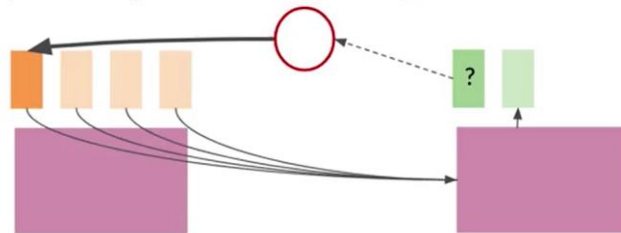
随着序列大小的增加，这会导致模型性能下降。因此，一个固定大小的编码器会存

在一个隐藏状态的问题，即它很难压缩更长的序列，最终限制了自己并惩罚了解码器。
谁能想出一个好的预测，如何做对解码器更好？

解决想法：

Solution: focus attention in the right place

- Prevent sequence overload by giving the model a way to focus on the **likeliest** words at each step
- Do this by providing the information specific to each input word



you give the model a way to focus its
attention in the right place at each step.



deeplearning.ai

2.2 对齐

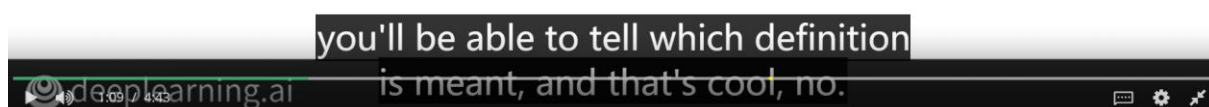
对齐在今天仍然广泛用于单词义消歧或词义发现。

那么从某种意义上说，您希望输入的单词对齐输出的单词。

Motivation for alignment

Correctly aligned words are the goal:

- Translating from one language to another
- Word sense discovery and disambiguation
- Achieve alignment with a system for retrieving information step by step and scoring it



请注意，英语翻译的单词比德语的单词多。执行单词对齐时，您的模型需要能够识别单词之间的关系，防止产生单词乱序或翻译不正确。为了能够正确对齐单词，您需要添加一层以帮助解码器了解哪些输入对每个预测更重要。

输入注意层，该注意层将执行以下一系列计算：分配了一些输入，比其他分配更大的权重。您可以在此处看到以深橙色显示的隐藏状态具有较重的线条，这表明它获得了更高的关注分数。这意味着解码器输出中的下一个单词将是受此编码器影响的隐藏状态。现在您知道什么是单词对齐，注意和通过给每个单词向量一个分数来计算对齐。

试题

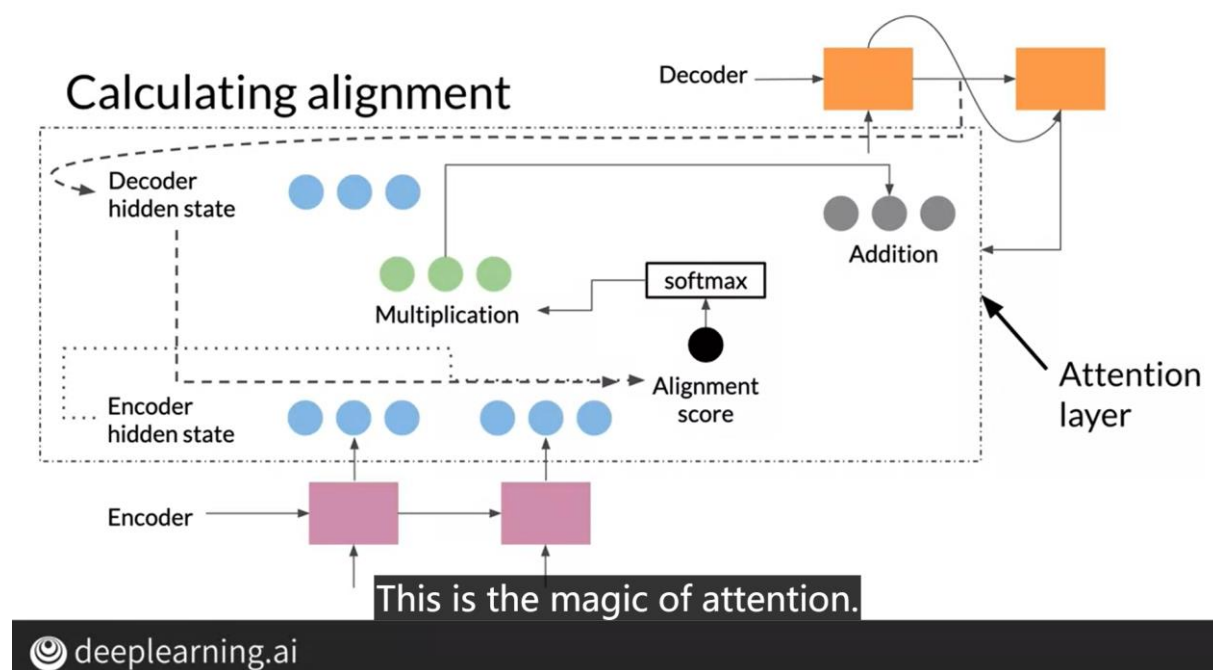
What happens when the attention mechanism assigns a higher attention score to a certain word in the sequence?

- ☐ The next word in the decoder's output will not be influenced at all by this word.
- ☐ The next word in the decoder's output will be weakly influenced by this word.
- ☒ The next word in the decoder's output will be strongly influenced by this word.

✓ 正确

Attention 流程简介：

首先，为编码器准备好所有可用的隐藏状态，并对解码器的第一个隐藏状态执行相同的操作。在简化的例子中，有两个编码器隐藏状态和一个解码器隐藏状态。接下来，通过获取每个编码器状态和解码器隐藏状态之间的点积，对每个编码器隐藏状态进行评分。如果其中一个分数高于其他值，则意味着此隐藏状态将比其他状态对输出产生更大的影响。然后你将通过 softmax 运行分数，所以每个分数都被转换成一个 0 到 1 之间的数字，这就给了你注意力的分布。取每个编码器的隐藏状态，乘以它的 softmax 分数，即 0 到 1 之间的数字，这就得到了对齐向量。差不多了，现在把对齐向量中的所有元素加起来，得到所谓的上下文向量。这家伙就是你输入解码器的东西，所以你可以看到这里发生的事情可以被提炼成一些数学运算，这些运算是根据单词的重要性来评分的。这就是注意力的魔力。下一步你会得到更深层的理解。所以你现在已经学会了对齐，你也看到了注意力是如何利用这个概念的。



您在上一门课程中学习的模型（RNN，LSTM，GRU）的顺序性质不允许在训练示例中进行并行化，这在较长的序列长度上变得至关重要，因为内存限制了示例之间的批处理。换句话说，如果您依赖序列，并且需要先了解文本的开头，然后才能计算文本的结尾，那么就不能使用并行计算。您将不得不等到初始计算完成。这不是很好，因为如果文本太长，则 1）处理该文本将花费很长时间，并且 2）接近结尾时，您将丢失文本前面提到的大量信息。

因此，注意力机制对于各种任务中的序列建模变得至关重要，允许对依赖项进行建模而不必过多关心它们在输入或输出序列中的距离。

在本课程中，您将学习这些注意力机制，并了解它们是如何实现的。

PS：这是来自《大西洋》的一篇文章，其中讨论了著名的肯尼迪（JFK）演讲，其中包含“对齐柏林（Ich bin ein Berliner）”一词，这是您在 Alignment 视频中看到的示例。

<https://www.theatlantic.com/magazine/archive/2013/08/the-real-意义-of-ich-bin-ein-berliner/309500/> （Putnam, 2013 年）

2.3 Attention

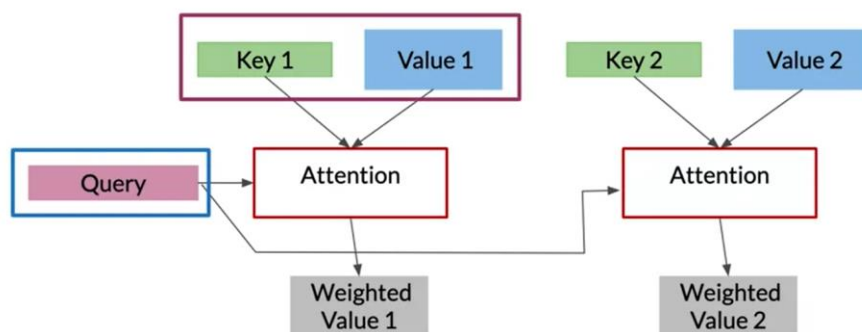
本节将讲解 Attention 的原理。

回顾所有的新概念，注意力是一层计算，它让你的模型集中在每一步的序列中最重要的部分。查询、值和键是编码器和解码器隐藏状态的表示。它们通过计算解码器查询和编码器键值对之间的相似度来检索注意力层内的信息。

简而言之，注意力就是这样做的。执行一个查询，选择查找键的可能性最高的位置，然后查找该键。注意机制使用输入或编码器隐藏状态和输出或解码器隐藏状态的编码表示。键和值是成对的。两者都是维数 N ，其中 N 是输入序列的长度，来自编码器的隐藏状态。键和值有各自的矩阵，但矩阵的形状相同，而且通常是相同的。当查询来自解码器的隐藏状态时，键值对和查询都从模型的两端进入注意层。一旦它们在里面，查询和键的点积就被计算出来了。这本质上是它们之间相似性的度量，相似向量的点积往往具有更高的值。给每个值的加权和由键与查询匹配的概率决定。

想象一下，您正在将英语翻译成德语。您可以将英语中的单词嵌入表示为键和值。然后，查询将与德语等效。然后，您可以计算查询和键之间的点积。请注意，相似的向量具有较高的点积，非相似的向量具有较低点积。直觉是您要在查询中标识与关键字相似的相应单词。这将使您的模型在翻译每个单词时“看起来”或专注于正确的位置。使用了 softmax 之后，这使我们能够得到 0 到 1 之间的数字分布。还需要一步。然后，我们将输出乘以 V 。记得 V 在此示例中，与我们的键相同，对应于英语单词 embeddings。这告诉我们每个单词有多少，或者我们将把哪些单词组合输入到解码器中以预测下一个德语单词。这称为标度点乘积注意。请注意，我们稍后会向其添加标准化常数，但是您现在不必担心这一点。

Inside the Attention Layer



This so
deeplearning.ai is called scale dot product attention.

试题

Keys and values are pairs, both coming from the encoder hidden state, while queries come from the decoder hidden states.

- ☒ True
☐ False

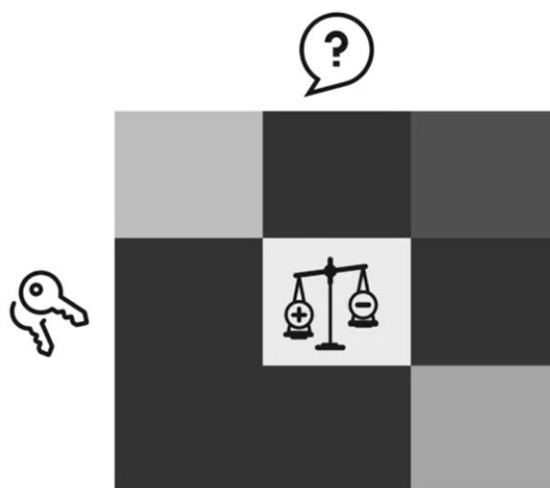
✓ 正确

Attention

Keys and queries = 1 matrix
with the words of one query (Q)
as columns and the words of the
keys (K) as the rows

Value score (V) assigned based on
the closeness of the match

Attention = $\text{Softmax}(QK^T)V$



but don't worry,

deeplearning.ai

灵活的匹配对齐:

Flexible attention

For languages with different grammar structures, attention still looks at the correct token between them

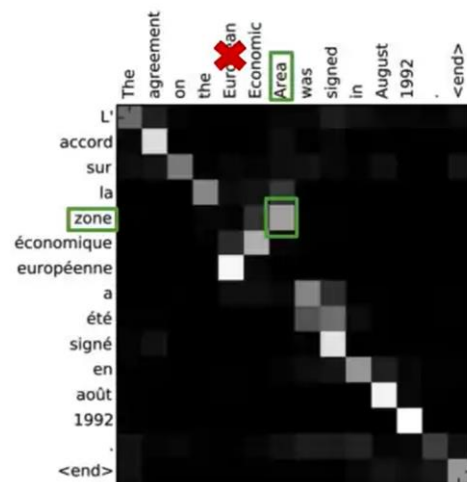


image ©
(Bahdanau
et al., 2015)

It's pretty amazing,

deeplearning.ai was a little matrix multiplication can do.

2.4 本章所用数据集

数据集：

Data in machine translation

English	German
I am hungry!	Ich habe Hunger.
...	...
I watched the soccer game.	Ich habe das Fußballspiel gesehen.

Attention! (no pun intended) Assignment dataset is not as squeaky-clean as this example and contains some Spanish translations.



deeplearning.ai

entirely clean as there are
occasional inputs in Spanish.

机器翻译的设置：

Machine translation setup

State-of-the-art uses pre-trained vectors

Otherwise, represent words with a one-hot vector to create the input

Keep track of index mappings with word2ind and ind2word dictionaries

Use start-of and end-of sequence tokens:



SOS, and EOS.



deeplearning.ai

Preparing to Translate to German

ENGLISH SENTENCE:

Both the ballpoint and the mechanical pencil in the series are equipped with a special mechanism: when the twist mechanism is activated, the lead is pushed forward.

TOKENIZED VERSION OF THE ENGLISH SENTENCE:

```
[4546 4 11358 362 8 4 23326 20104 1745 8210 9641 5 6
4 3103 31 2767 30 13 914 4797 64 196 4 22474 5 4797 16
24864 86 2 4 1060 16 6413 1138 3 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```



deeplearning.ai

a matrix for the
embedding vectors.

English to German

GERMAN TRANSLATION:

Der Kugelschreiber und der Drehbleistift der Serie sind mit einem besonderen Mechanismus ausgestattet: Bei Betätigung der Drehmechanik wird die Schreibmine nach vorne geschoben.

TOKENIZED VERSION OF THE GERMAN TRANSLATION:

```
[ 149 3892 5280 14774 2418 12 11 9883 6959 7298 15157 5 11 8453 75
39 114 5324 10565 2520 64 752 12954 26538 147 11 9883 23326 20104
300 78 10 21150 10166 126 14566 5 23850 1171 3 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```



deeplearning.ai

of unpaired tokens
in the sequence.

这些令牌稍后将用于产生 嵌入向量的矩阵。

2.5 用 Attention 训练 NMT

● Teaching force

如何检验自己的翻译预测文本是否是正确的呢？我们不能单纯的用每个词与实际期望文本进行对比验证（毕竟同样意思的句子可以有不同的表示），我们需要一种方法，可以检查每个步骤所做的工作（因为在某一步出现错误的话，后续步骤正确的可能性会很小）。

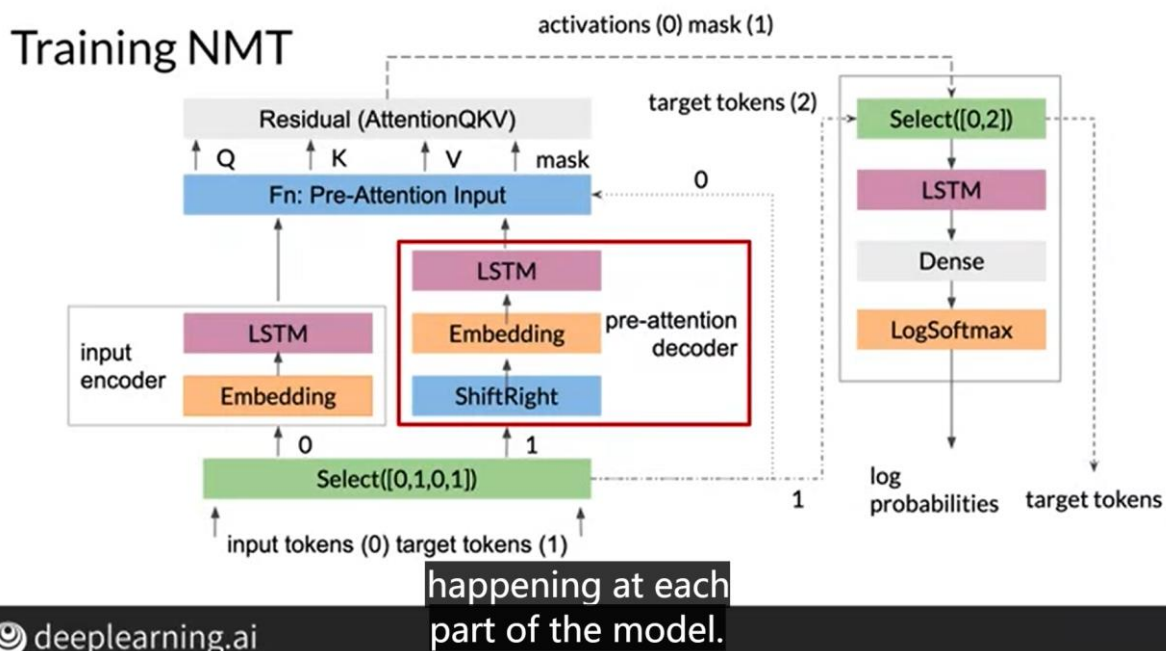
一个重要结论是，在训练期间，预测的输出不用于预测下一个预测的绿色矩形。相反，实际输出，或基本真实值，是每个时间步的输入，直到序列结束。如果没有教师的强迫，模型的收敛速度可能会很慢。

试题

What is the purpose of using teacher forcing?

- ☒ Teacher forcing provides faster training and higher accuracy by allowing the model to use the decoder's actual output to compare its predictions against
- ☐ Teacher forcing works by forcing the model to punish itself when it gets things wrong.
- ☐ Teacher forcing is not useful and therefore not often used.

✓ 正确

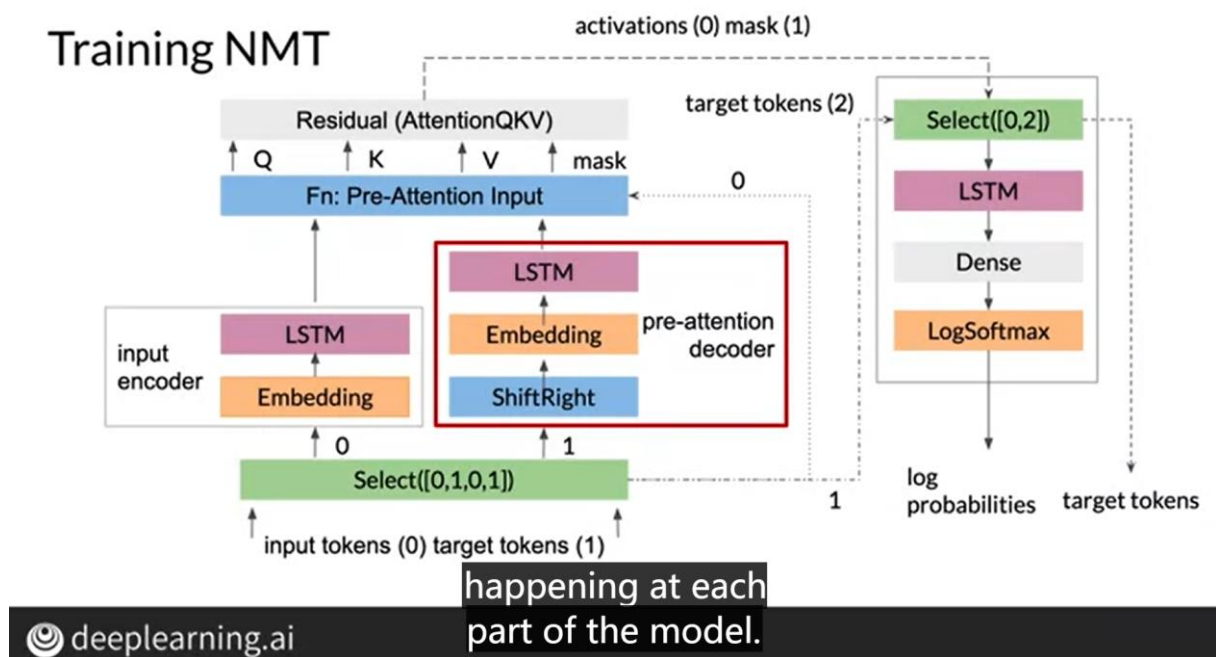


整个流程的讲解翻译如下（较为重要）：

初始选择生成两个副本。每个输入标记由零表示，目标标记由 1 表示。请记住，这里的输入是英语令牌，目标是德语令牌。输入令牌的一个副本被输入到输入编码器中，以转换成键和价值向量。而目标令牌的副本进入预注意解码器。这里需要注意的是，pre-attention 解码器并不是前面展示的解码器，产生解码后的输出。预注意解码器将预测目标转换成另一个称为查询向量的向量空间。这将计算相对权重以给出每个输入权重。预注意解码器获取目标标记并将其向右移动一个位置。这就是 Teaching force 的地方。每一个标记将向右移动一个位置，并且在句子的开头，标记将是每个序列开头的符号。

接下来，输入和目标被转换成单词的嵌入或初始表示。现在已经有了查询、键和价值向量，可以为注意力层准备它们了。您还将应用填充掩码来帮助确定填充标记。在计算 Q, K 转置后使用掩模。在计算 softmax 之前，编程赋值中的 where 运算符将把零填充标记转换为负 10 亿，在计算 softmax 时，这个值将近似为零。这就是填充的原理。

现在，所有的事情都准备好了，在里面所有分配权重的计算都会发生。剩余块将在预注意解码器中生成的查询添加到注意层的结果中。然后注意力层输出它的激活和先前创建的掩模 (mask)。在通过解码器运行所有内容之前，是时候放下掩模 (mask) 了，这就是第二个 Select 正在做的事情。它从注意层或零，以及目标标记的第二个副本，或两个激活。你会记得很久以前的事吗。这些是解码器需要与预测进行比较的真实目标。快完成了。然后通过一个密集层或一个简单的线性层来运行所有内容。这将为您的输出提供正确的大小。最后，您将获得输出并通过 LogSoftmax 运行它，它将注意力权重转换为 0 和 1 之间的分布。最后四个步骤组成了你的解码器。真正的目标标记仍然在这里，我们将传递与预测匹配的日志概率。这就是你要建立的模型，以及步骤背后的直觉。



这里附上阅读材料中对此图的讲解：

在此示例中，我们将英语标记转换为德语标记。输入用 0 表示，目标用 1 表示。输入令牌的一个副本输入到输入编码器中，然后转换为键和值向量。目标令牌的一个副本进入预注意解码器。这里需要注意的重要一点是，预注意解码器不是您先前显示的解码器，它会生成解码的输出。注意前解码器将预测目标转换为称为查询向量的不同向量空间。

在本周以前，我谈到了使用 $\text{softmax}(QK^T)V$ ，预注意解码器为您提供 Q 向量。

更具体地说，预注意解码器获取目标令牌并将其右移一位。这是老师强迫的地方。每个令牌都将向右移一个位置。这样，当我们预测时，我们可以只输入正确的目标单词（即老师强迫）。

输入编码器为您提供键和值。一旦获得查询，键和值，就可以计算注意力。注意，在上图中，我们对填充的令牌使用了掩码。作为技术细节，在分配中，将通过将填充的令牌转换为 -1 十亿来完成填充。当您计算其 softmax 时，您将获得 0。为什么会这样呢？（请随时停止并对此说服自己）。

在获得注意层的输出之后，残差块会将将在注意前解码器中生成的查询添加到注意层的结果中。然后激活进入第二阶段，使用先前创建的遮罩。我们现在位于图像的右上角。选择用于放置遮罩。它从关注层或 0，以及目标令牌的第二个副本或 2，进行激

活。这些是解码器需要与预测进行比较的真实目标。

最后，只需通过解码器 LSTM / dense 层或目标对象的 vocab 大小的简单线性层运行所有内容。这样可以为您的输出提供合适的尺寸。我们使用 log softmax 来计算概率。真正的目标代币仍在这里闲逛，我们将把与预测值匹配的对数概率传递给其他人。

注：需要更多 Teaching force 的知识，可以参考如下链接：

<https://towardsdatascience.com/what-is-teacher-forcing-3da6217fed1c?gi=90b40382e2b8>

2.6 机器翻译评估 (NMTE)

● BLEU

通常，BLEU 分数越接近 1，您的模型越好。接近零，情况越糟。话虽如此，什么是 BLEU 分数，为什么这是重要指标？简而言之，就是选定了 N-gram 的 N 之后，统计 MT 中切割文本（每个都是 N 个单词）与人类翻译文本（reference）的重合比例，越接近于 1 越好，但这种方法对较短的翻译具有偏好性。因此，需要加上一定的惩罚优化才具有普适性。可参考这篇文章的总结：[BLEU 总结](#)

缺点：没有考虑语义，也没注意语句结构（即词语的先后顺序）。

● ROUGE

它代表的是面向召回的 Gisting Evaluation 的替补，在默认情况下更倾向于召回。这意味着它更加重视在机器翻译中出现了多少人为创建的引用。ROUGE 最初是用来评估机器摘要文本的质量的，但也可以用于评估机器翻译。它的工作原理是将机器文本或系统文本与通常由人类创建的参考文本进行比较。ROUGE 评分通过计算机器文本和参考文本之间的 n-gram 重叠来计算机器文本的精确度和召回率。

Recall in ROUGE

Model	The	cat	had	striped	orange	fur
Reference	The	cat	had	orange	fur	

$$\frac{(\text{Sum of overlapping unigrams in model and reference})}{(\text{total \# of words in reference})} = \frac{5}{5} \text{ Recall} = 1$$

But what does that
actually tell you?

Precision in ROUGE

Model	The	cat	had	striped	orange	fur
Reference	The	cat	had	orange	fur	

(Sum of overlapping unigrams in model and reference)

(total # of words in model)

$\frac{5}{6}$ Precision = 0.83

or roughly 83 percent of the words were relevant.

ROUGE 的缺点：低分数并不意味着翻译的效果差，因为 ROUGE 本来就是用于文章摘要评估的，更倾向于召回的判定。感觉和 BLEU 差异不大，见本节最后一图。BLEU 是将一个 MT 和多个人类的 reference 进行对比，ROUGE 是将一个 MY 与目标翻译进行打分对比（个人理解）。

Problems in ROUGE

- Doesn't take themes or concepts into consideration (i.e., a low ROUGE score doesn't necessarily mean the translation is bad)

Model	I	am	a	fruit-filled	pastry
Reference	I	am	a	jelly	donut



These are just a couple of caveats to keep in mind as

总结：

Summary

- BLEU score compares “candidate” against “references” using an n-gram average
- BLEU doesn’t consider meaning or structure
- ROUGE measures machine-generated text against an “ideal” reference



By taking the average of

2.7 采样与解码

本节讲两种方法，构建翻译后的句子。 第一种方法称为贪婪解码， 第二种方法称为随机抽样。还会介绍以下知识，抽样中的技巧（温度），束搜索和最小 Bayes 风险。

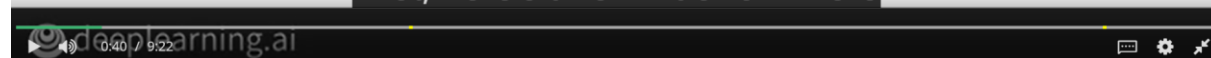
分享

Outline

- Random sampling
- Temperature in sampling
- Greedy decoding
- Beam search
- Minimum Bayes' risk (MBR)



First, here's a reminder of where



● 贪婪解码

Greedy decoding

Selects the most probable word at each step

But the best word at each step may not be the best for longer sequences...

Ich habe Hunger.

I am hungry.

I am, am, am, am...

gives you I am, am, am, am.



贪婪解码是最简单的解码方式 选择模型时的预测 每个步骤中最可能出现的单

词。但是，这种方法有局限性。假设考虑每个预测的最高机率对应的单词，串联所有预测单词之后的输出序列，往往不是合适的答案。

- 随机采样

Random sampling

am	full	hungry	I	the
0.05	0.3	0.15	0.25	0.25

Often a little too random for accurate translation!

Solution: Assign more weight to more probable words, and less weight to less probable words.

and less weight to the others.



随机抽样的作用是提供每个单词的概率，并相应采样以获取下一个输出。问题之一可能是有点太随意了。一个解决方案是为具有较高概率的单词分配更多权重，减轻别人的负担。

- 温度

温度是一个参数，用来调节随机采样中预测的随机性，测量范围是 0 到 1。温度值越高，代表你的选择越大胆，很有可能会得到一些有趣、疯狂的答案，但这往往也意味着错误率会更高。

试题

Say you wanted to build an NMT model that makes very weird and funny, but possibly wrong, predictions. Would you set your temperature parameter higher, or lower?

☒ Higher!

☐ Lower!

✓ 正确

- Beam search decoding

不像贪婪解码那样，仅提供一个最高概率输出，Beam search decoding 使用广度优先构建搜索流，基于条件概率搜索选择多个选项。光束宽度参数 B，限制了分支路径数量。

Beam search decoding

A broader, more exploratory decoding alternative

Selects multiple options for the best input based on conditional probability

Number of options depends on a predetermined beam width parameter B

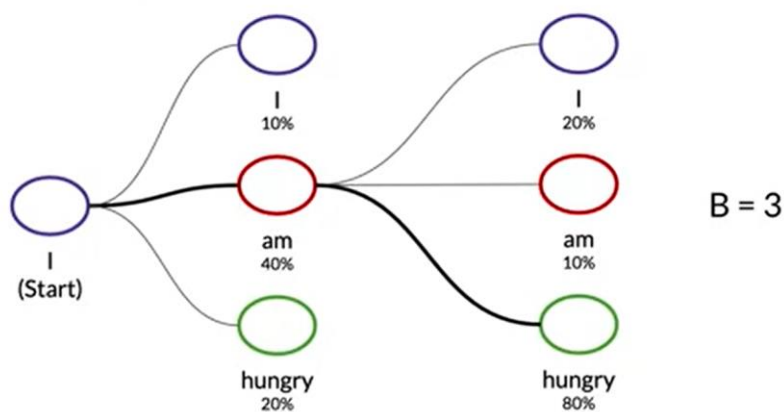
Selects B number of best alternatives at each time step



deeplearning.ai

to select several
possible options.

Beam search example



deeplearning.ai

machine translation
of longer sequences.

缺点：

该模型学习的分布在现实中是没有用或不准确的。它还可能会以有问题的方式使用单个令牌，特别是对于不干净的语料。如下图：

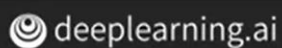
Problems with beam search

“Ich mag die Vereinigten Staaten, weil die Vereinigten Staaten groß sind.”

Even with 11 good English translations of “Vereinigten Staaten,” but a ~1% probability of the non-word “Uhm” occurring, you might get this as a translation:

“I like the United States, because the Uhm is big.”

Even with 11^2 good translations, the most probable one will still be “Uhm.”



Well, thankfully, there are
alternatives to consider.

- 最小 Bayes 风险

Minimum Bayes Risk (MBR)

Compares many samples against one another. To implement MBR:

- Generate several random samples
- Compare each sample against all the others and assign a similarity score (such as ROUGE!)
- Select the sample with the highest similarity: the golden one ✨



which is sometimes referred
to as the golden one.

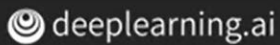
MBR 步骤:

Example: MBR Sampling

To generate the scores for 4 samples:

1. Calculate similarity score between sample 1 and sample 2
2. Calculate similarity score between sample 1 and sample 3
3. Calculate similarity score between sample 1 and sample 4
4. Average the score of the first 3 steps (Usually a weighted average)
5. Repeat until all samples have overall scores

You'll be implementing
this one in



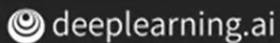
本节总结:

Summary

- Beam search uses conditional probabilities and the beam width parameter
- MBR (Minimum Bayes Risk) takes several samples and compares them against each other to find the **golden one** ✨
- Go forth to the coding assignment!



compares them against themselves,



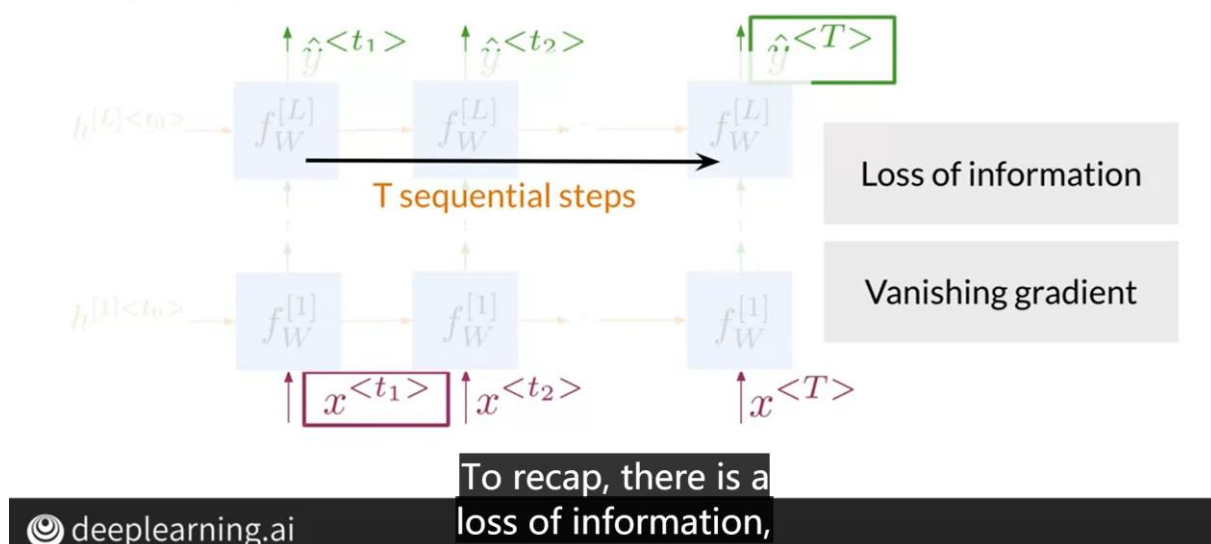
3. 第二周

3.1 Trasformer 和 RNN

使用 RNN，您必须采取顺序步骤对您的输入进行编码，从输入开始就开始每一步的计算，直到到达终点。

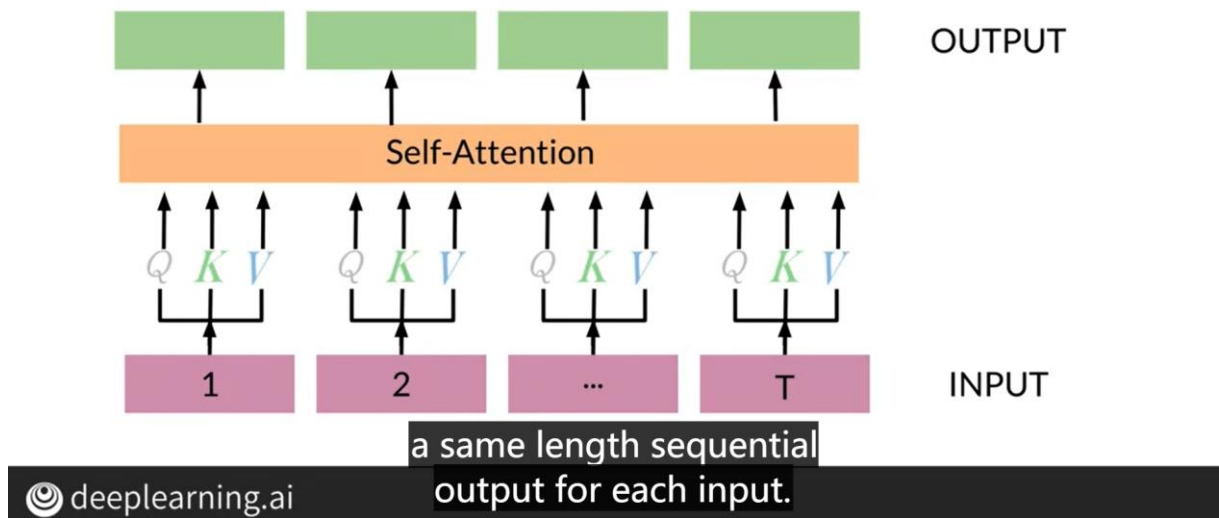
你必须经历每一个字在您的输入中以第一个词后跟第二个单词，一个接一个。在顺序匹配器中，为了开始翻译，这也是按顺序进行的。因此，没有这里有很大的并行计算空间。输入序列中包含的字词越多，处理该句子所花费的时间越多，而且随之带来的问题有信息丢失和梯度消失。

Seq2Seq Architectures



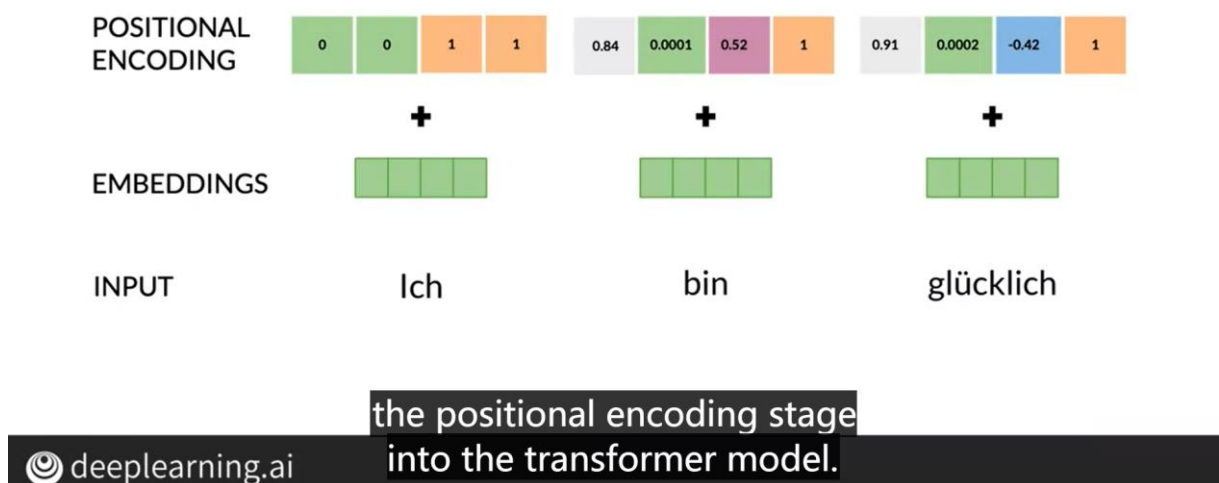
通过使用多头注意层而不是重复层，变压器因序列而异。要理解多头注意力的直觉，我们先回顾一下自我注意。Self-attention 直接使用从每个连续输入中获得的查询、键和值。最后，它为每个输入输出相同长度的顺序输出。自我注意可以理解为一个意图模型，它为每个输入、查询、键和值都包含一个密集层。现在你可以考虑添加一组平行的自我注意层，这些层被称为头部。

RNNs vs Transformer: Multi-headed attention

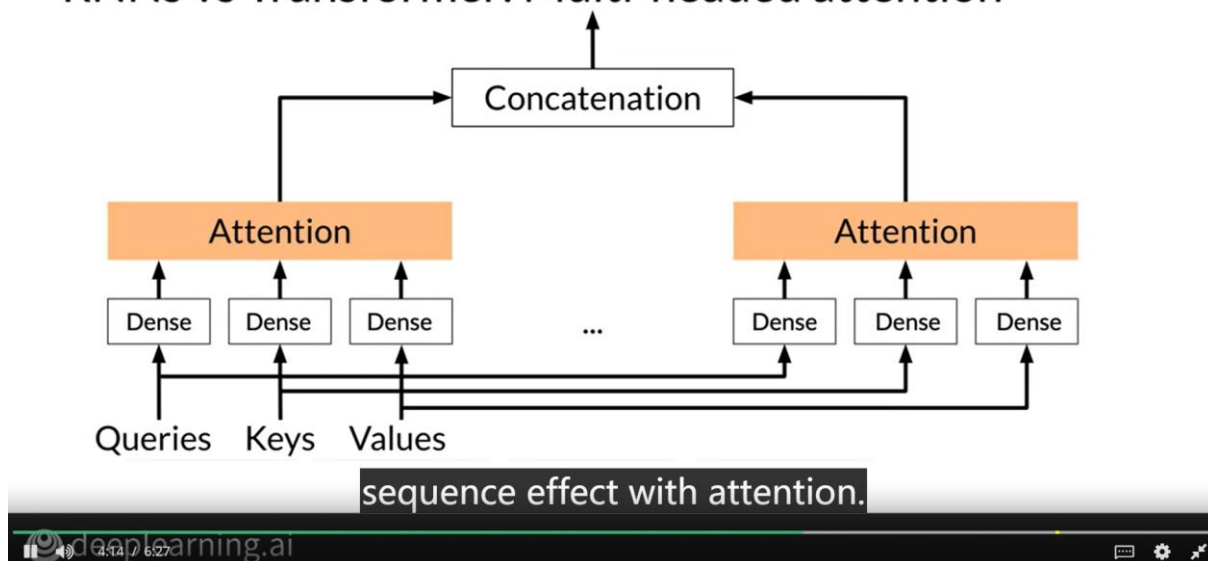


Transformer 还使用位置编码来保留输入序列的位置信息。位置编码输出将要添加到嵌入中的值。这就是给模型的每个输入词都提供一些关于它的顺序和位置的信息。

RNNs vs Transformer: Positional Encoding



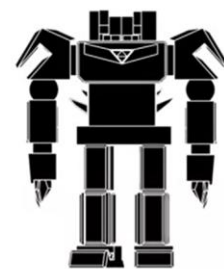
RNNs vs Transformer: Multi-headed attention



优缺点总结:

Summary

- In RNNs parallel computing is difficult to implement
- For long sequences in RNNs there is loss of information
- In RNNs there is the problem of vanishing gradient
- Transformers help with all of the above



But fortunately recent research has found ways to

与递归层不同的是，多头注意力层独立地计算序列中每个输入的输出，然后允许我们进行并行计算。但是它不能为给定的序列建立序列信息的模型。这就是为什么需要将位置编码阶段合并到转换器模型中。总之，RNN 有一些问题是由它们的序列结构引起的。使用 RNNs，很难充分发挥并行计算的优势。对于长序列，重要信息可能会在网络中丢失，并出现梯度消失问题。但幸运的是，最近的研究发现了使用变压器来解决 RNN 的缺点的方法。变压器是 RNN 的一个很好的替代品，它有助于克服 NLP 和许多处理

序列数据的领域中的这些问题。

课后测验：

试题

Select every true statement.

☒ Even RNN architectures like GRUs and LSTMs don't work well for really long sequences.

✓ 正确

☒ Transformers are able to take more advantage from parallel computing than RNN architectures.

✓ 正确

☒ With transformers, the vanishing gradient problem isn't related with length of the sequences.

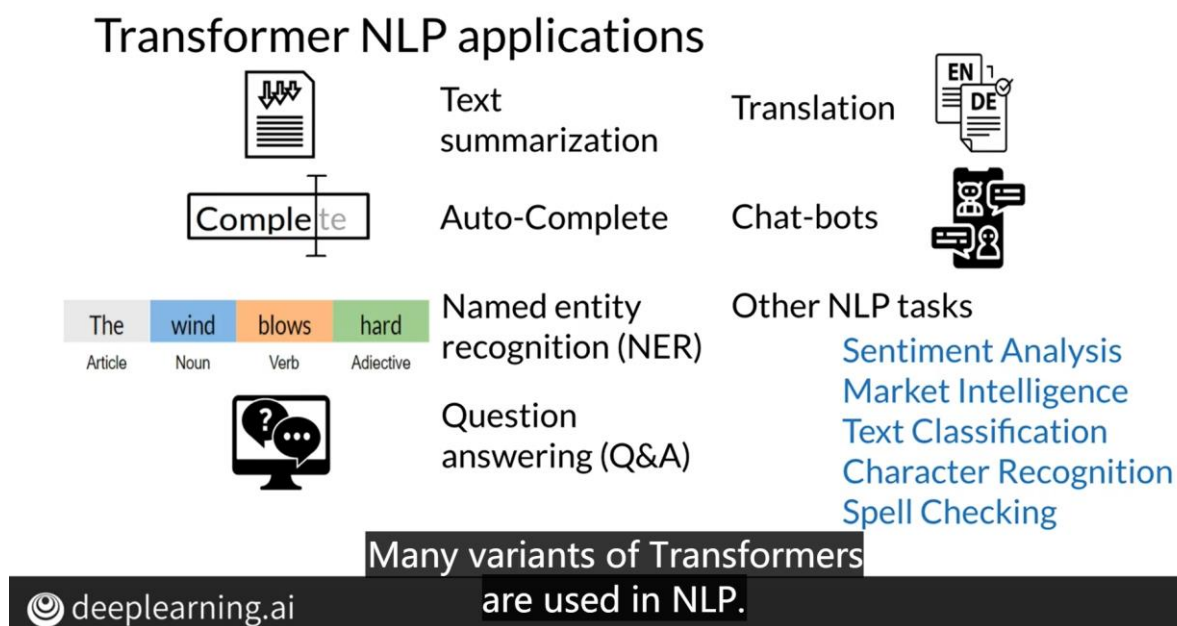
✓ 正确

☐ Transformers are models that use both recurrent units and attention mechanisms.

3.2 Transformer 的应用

- Transformer 在 NLP 中的应用
- 一些 Transformer 的介绍
- T5 的简介 (Text-to-Text Transfer Transformer, 简称 T5)

Transformer 通常适用于任何顺序型任务, 它已经在整个 NLP 中应用广泛。当前一个非常流行的应用是 自动提取文本摘要 (automatic text summarization.)。



一些 Transformer 的介绍:

State of the Art Transformers

Radford, A., et al. (2018)
Open AI

Devlin, J., et al. (2018)
Google AI Language

Colin, R., et al. (2019)
Google

GPT-2: Generative Pre-training for
Transformer

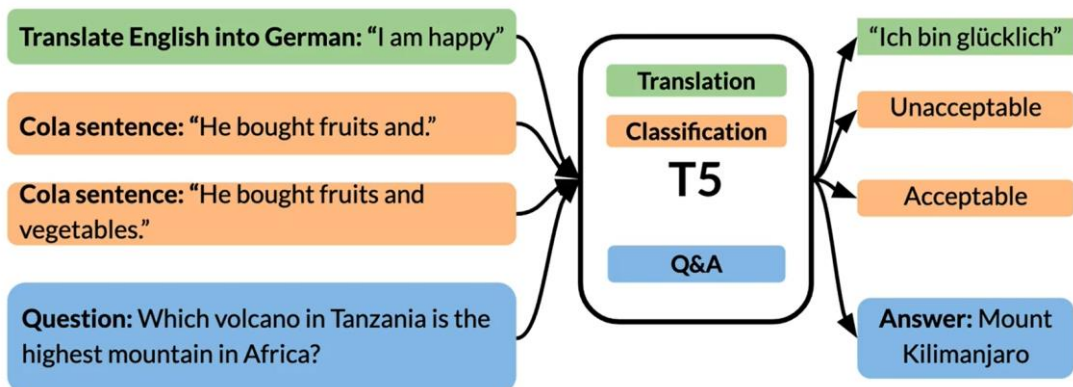
BERT: Bidirectional Encoder
Representations from Transformers

T5: Text-to-text transfer transformer

deeplearning.ai this can do question answering
among the lots of different tasks.

T5 的应用，可以一体化执行翻译、分类和 QA 聊天：

T5: Text-To-Text Transfer Transformer

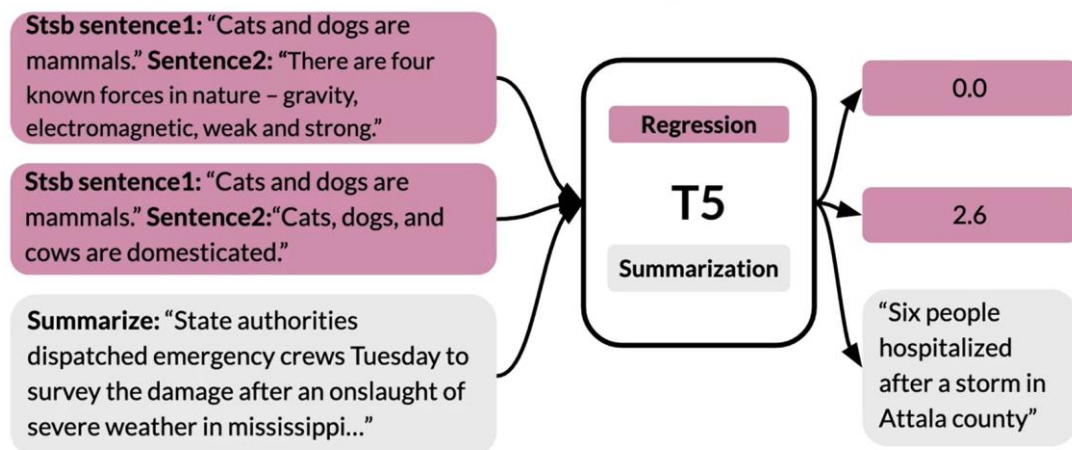


How cool is that?

deeplearning.ai

T5 还能够执行回归和汇总任务：

T5: Text-To-Text Transfer Transformer



deeplearning.ai **six people hospitalized after a storm in Attala County.**

所以，问题来了，Transformers 能够同时运行一些 NLP 任务吗？

试题

Can transformers perform several NLP tasks simultaneously?

- ☒ Yes
- ☐ No

✓ 正确

That's right! Because you can train a single model that is able to perform multiple NLP tasks together.

一个模型可以处理如此多种任务不是很令人惊讶吗？原理呢？让我们看下一节。

3.3 点乘注意 (Dot-Product Attention)

本节会教您如何使用 Attention，以及相应的较为复杂的 Attention 变体。

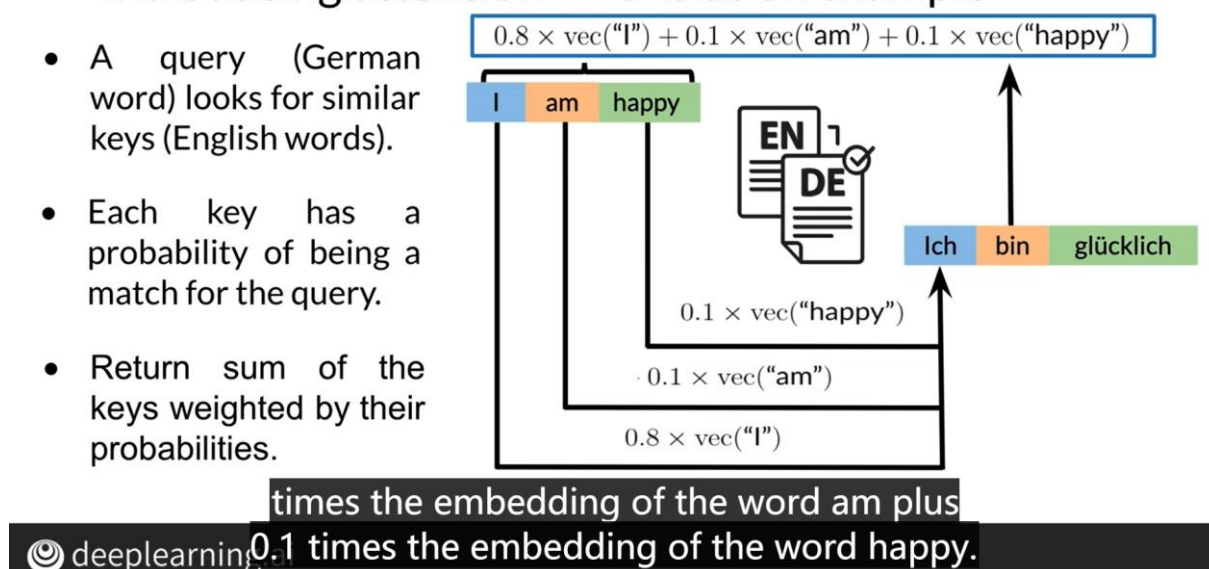
- 介绍 Attention
- Attention 背后的数学思维

首先举例介绍注意力模型的想法，以英语翻译为德语为例。

当你有一个德语单词时，你要看看英语句子，找出所有有相似单词的地方。这就是注意力对你的作用。在注意层，德语单词向量称为查询。因为它们启动了查找，所以一个查询与所有键匹配，每个人都有可能匹配该查询。例如，要翻译这个句子，“”我很高兴“”（德语），你可以把英语中的 I 和下面的 I-c-h, Ich 进行比较。然后你可以比较 am 和 Ich，最后你可以比较 happy 和 Ich。这些按键的查询矩阵通常称为注意权重，它表示每个键与每个查询的相似程度。例如，Ich 这个词的向量可能与 I 的向量相似，它的概率为 0.8，而 am 和 happy 的概率不是很相似，但还是有一点点关系，两者的概率都是 0.1。最后，查询得到概率等待的关键向量的总和。在这个例子中，这些向量的加权和是单词 I 的嵌入量的 0.8 倍加上单词 am 的嵌入量的 0.1 倍加上单词 happy 的嵌入量的 0.1 倍。

Introducing attention - Translation example

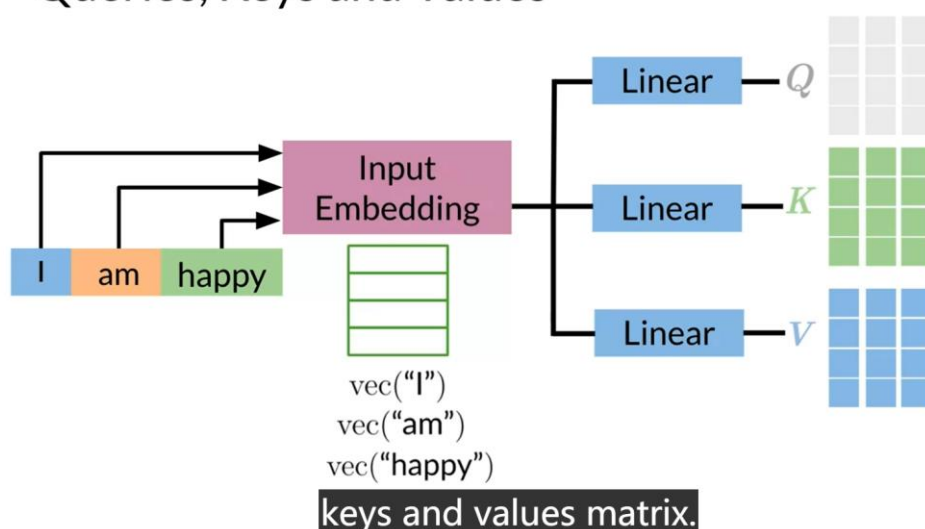
- A query (German word) looks for similar keys (English words).
- Each key has a probability of being a match for the query.
- Return sum of the keys weighted by their probabilities.



在深入研究注意力的概念之前，您需要首先定义三个主要矩阵，大写字母 Q 代表查询，大写 K 代表键，大写 V 代表值。所以了解如何创建这些矩阵。让我们考虑一下英语中的短语“我很高兴”。首先，嵌入单词 I，以获得一个向量表示，该向量表示为

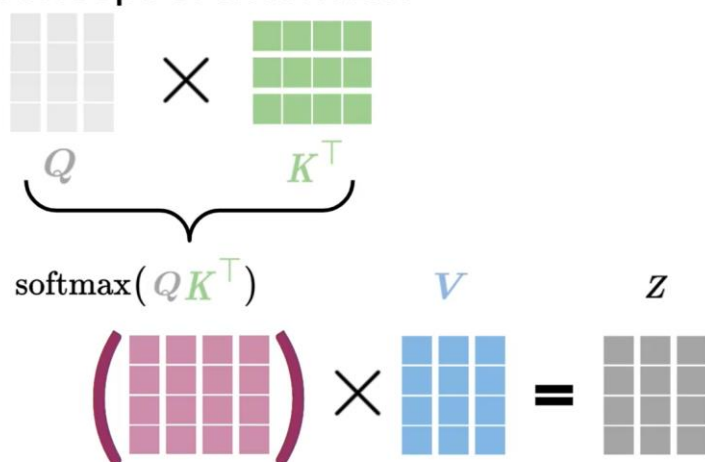
每个单词保存唯一的连续值，如本课程前面所述。通过提供三个不同的线性层，可以得到三个不同的查询向量、键和值向量。然后你可以对单词 am 做同样的操作来输出第二个向量。最后得到第三个词的快乐向量并形成查询、键和值矩阵。从大写 Q 矩阵和大写 K 矩阵和注意力模型计算权重或分数，表示特定查询的键的相对重要性。这些注意权重可以理解为对齐过程，因为它们来自 dot 产品。此外，要将这些权重转换为概率，需要一个 softmax 函数。

Queries, Keys and Values



最后，将这些概率与值相乘，您将获得一个加权序列，这就是注意力结果本身。

Concept of attention

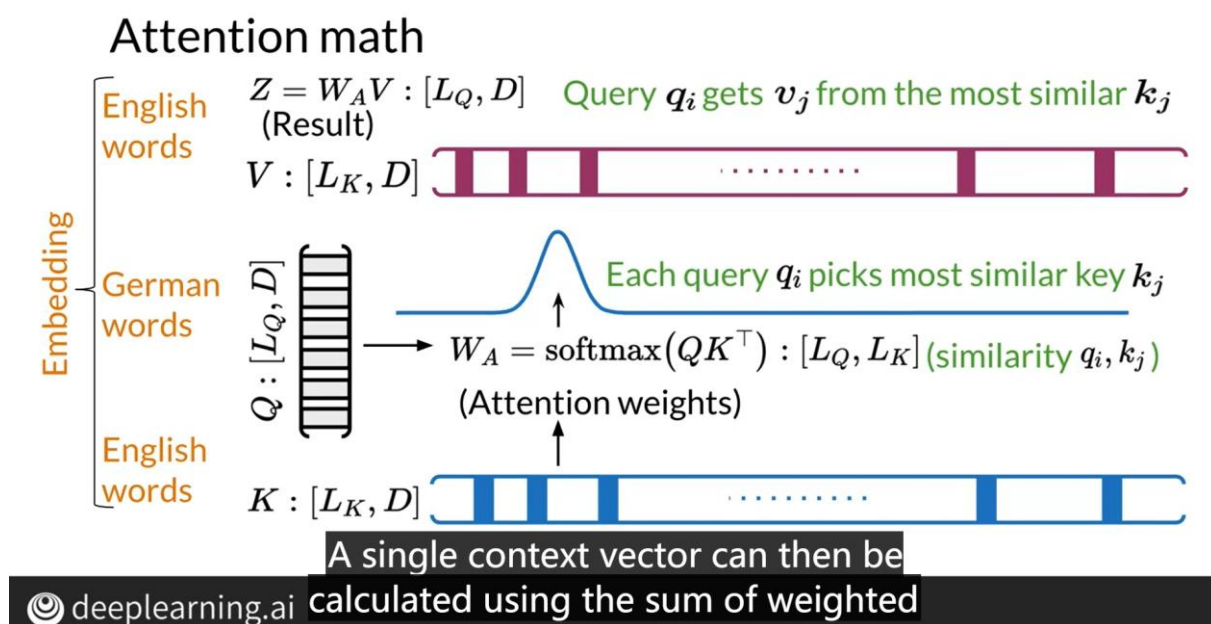


Finally, multiplying these probabilities with the values,

● Attention 背后的数学思想（三个矩阵的维数介绍）

Attention 的输入是查询（Query）、键（Key）和值（Value）。通常 value 与 key 相同。查询（Query）是维数为 T 的向量，并且有一个数量 L_Q 。键是具有相同维数 D 的向量，其中有一个 L_K 。把 key 看作是英语的词嵌入，把查询 Query 看作是德语的词嵌入，那么 D 是词嵌入的维度， L_K 是英语句子的长度， L_Q 是德语句子的长度。

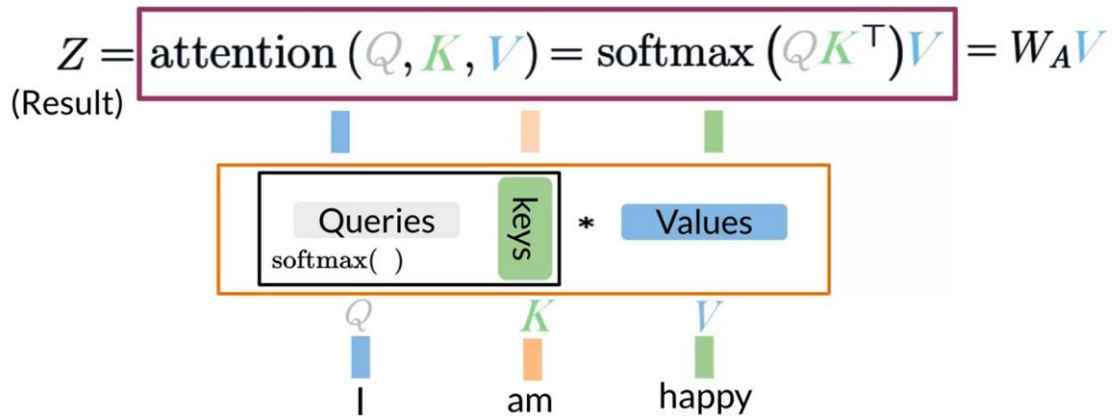
所以查询是形状 $L_Q \times D$ 的张量 Q ，而键的形状是 $L_K \times D$ 。正如我前面所说，查询 Q 会给每个键分配一个键 K 与 Q 匹配的概率，你可以用向量的乘积来度量相似度。所以如果 $Q \cdot K$ 很大，那么 Q 和 K 是相似的。然而，相似度加起来不等于一，所以不能用它们作为概率。要使其如此，并使注意力更集中在最佳匹配的按键上，请使用 softmax。所以你要计算查询键概率的矩阵，通常称为注意权重，就像 $Q \cdot K$ 转置之间的 softmax 一样，这个矩阵的维数是 $L_K \times L_Q$ 。



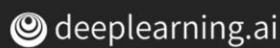
在获取值的最后一步中，每个查询都会获得一个概率，这是另一个与键形状相同的矩阵。通常与 keys 相同，我们希望得到一个加权和，通过 key k_i 匹配查询的概率加权每个值 v_i 。这可以非常有效地计算，就像矩阵乘法一样，我们将注意力权重 W_A 乘以值 V ，就这样。注意机制计算动态权重或对齐权重，表示序列中输入的相对重要性。（看图更容易理解）

直接上公式总结：

Attention formula



Q times K transpose and then by V and



once after softmax operation.

● 当堂总结

Summary

- Dot-product Attention is essential for Transformer
- The input to Attention are queries, keys, and values
- A softmax function makes attention more focused on best keys
- GPUs and TPUs is advisable for matrix multiplications



In general terms, the attention



takes as inputs, queries, keys and

● 课后测验

试题

Attention weights come from a dot product. A softmax function is needed to convert these weights into probabilities.

☒ True

☐ False

✓ 正确

Correct! It normalizes all scores to a probability distribution. In this way a good probabilistic interpretation of the attention weights is obtained, achieving that the attention is more focused on the best keys.

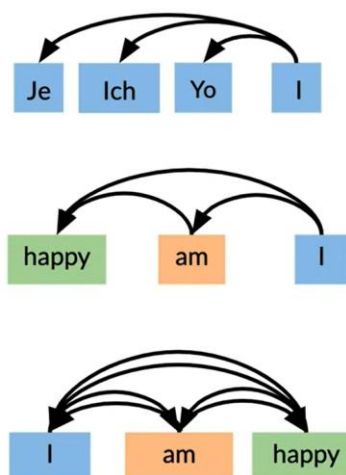
3.4 Casual Attention

- Attention 方法
- Casual Attention 概述
- Casual Attention 背后的数学思想

一共有三种 Attention 的方法：

Three ways of attention

- **Encoder/decoder attention:** One sentence (decoder) looks at another one (encoder)
- **Causal (self) attention:** In one sentence, words look at previous words (used for generation)
- **Bi-directional self attention:** In one sentence, words look at both previous and future words



This is used in models like BERTs and

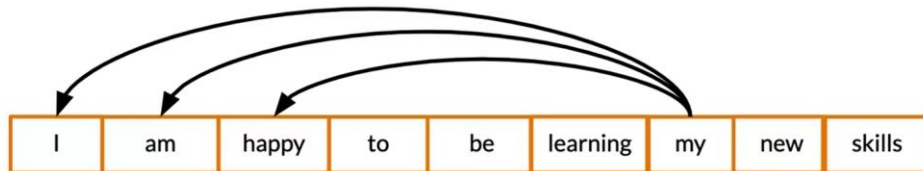


T5, that have a masking glass.

您已经看到了查询来自一个句子而键来自另一个句子的情况。在因果关系中，查询和键来自同一句子。这就是为什么它通常被称为自我关注。一般而言，因果注意可以生成之前与之相关的单词的单词（向前关注），但无法获得未生成单词的信息。

Causal attention

- Queries and keys are words from the same sentence
- Queries should only be allowed to look at words before



these were not generated yet, they can

deeplearning.ai attend to any word in the past though.

- Casual Attention 背后的数学思想

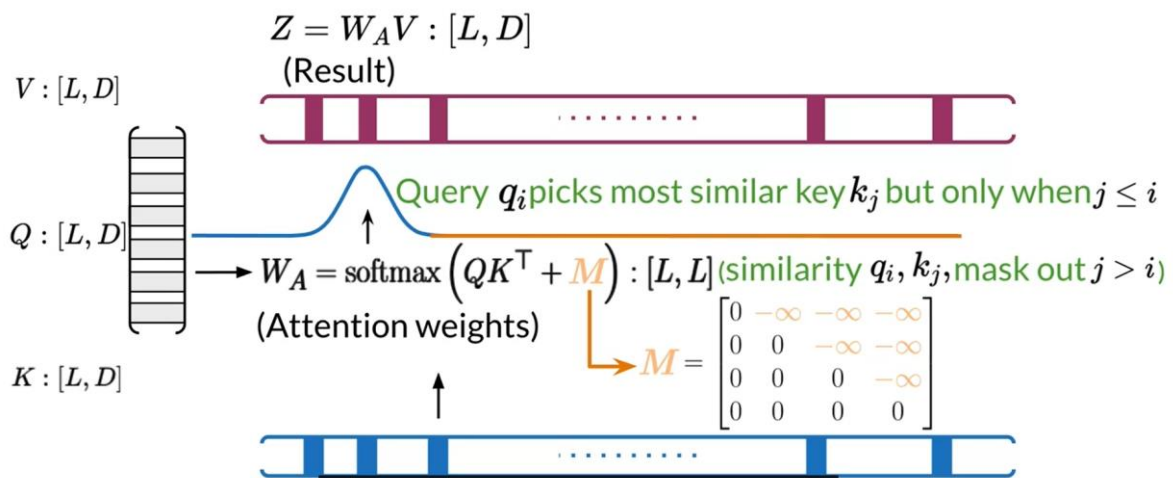
Causal attention math

→ Minus infinity -in practice, a huge negative number

$$\begin{array}{c}
 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} & \times & \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \\
 Q & & K^T \\
 \\
 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} & + & \begin{array}{|c|c|c|c|} \hline 0 & \square & \square & \square \\ \hline 0 & 0 & \square & \square \\ \hline 0 & 0 & 0 & \square \\ \hline 0 & 0 & 0 & 0 \\ \hline \end{array} & = & \\
 QK^T & & M & & \\
 \\
 \begin{array}{|c|c|c|c|} \hline 0 & \square & \square & \square \\ \hline 0 & 0 & \square & \square \\ \hline 0 & 0 & 0 & \square \\ \hline 0 & 0 & 0 & 0 \\ \hline \end{array} & + & \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} & = & \\
 M & & QK^T + IM & &
 \end{array}$$

deeplearning.ai

Causal attention math



is still very fast to compute, and

as you'll see it does the trick.

● 当堂测验

试题

Which allows the self-attention mechanism?

☒ Learn the dependencies between the words of the sentence.

✓ 正确

☐ It allows one sentence to learn from another and the words to look at previous words as future ones.

☒ Helps the encoder look at other words in the input sentence while encoding a specific word.

✓ 正确

☒ From embedded words a query, key and value vector is created by multiplying the embedded by three trained matrices.

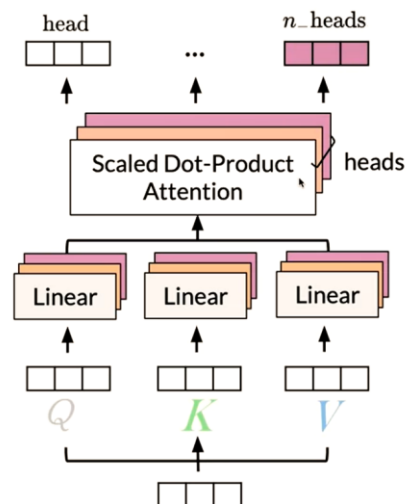
✓ 正确

3.5 multi-head attention

- multi-head attention 的思想启蒙
- 规模点积和其联系
- multi-head attention 的公式

Multi-Head Attention

- Each head uses different linear transformations to represent words
- Different heads can learn different relationships between words



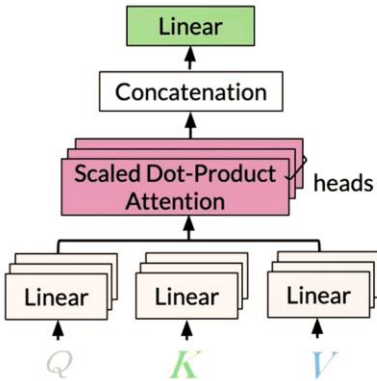
the heads are being concatenated,

矩阵的维数介绍：

输入 Q , K , V shape 都是：批次、大小、长度记为 D 。通常 D 是 512 或 1024。有时或多或少，但按此顺序。线性层作用于最终尺寸，并以 D 头将其更改为 N 头。 N 头是平行注意头的数量，通常是四个，八个甚至 16 个或更多。 D 头是每个头的向量维数，通常是 64 或 128。因此，线性层会创建一组 N 个头部向量，每个头的维数是 D 。

Multi-Head Attention - Concatenation

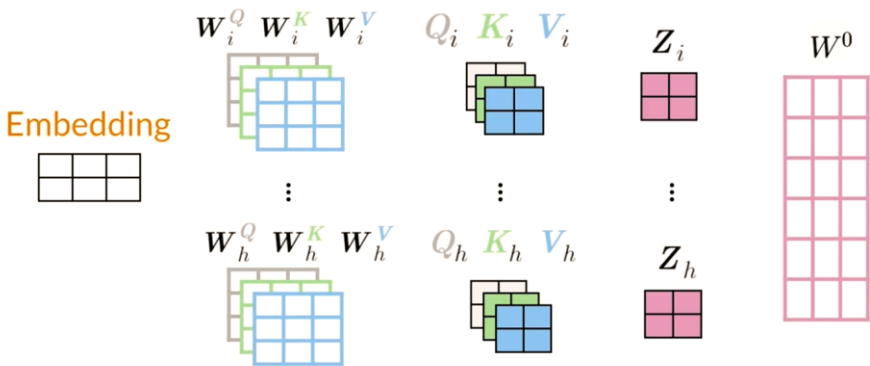
- Input(Q, K, V): [batch, length, d_model]
- Linear layer: [batch, length, n_heads * d_head]
- Transpose: [batch, n_heads, length, d_head]
- Apply attention treating n_heads like batch
- Result shape: [batch, n_heads, length, d_head]
- Transpose: [batch, length, n_heads * d_head]
- Linear layer into: [batch size, length, d_model]



different heads into a joint representation.

multi-head attention 的运作流程:

Multi-Head Attention math



you obtain the output layer Z .

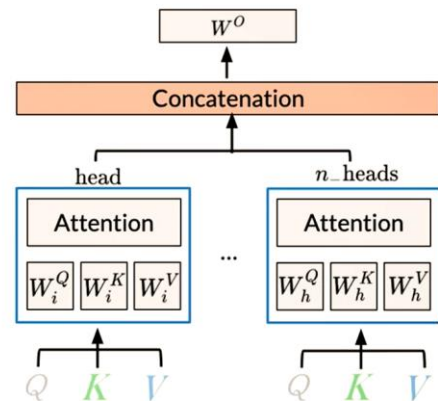
multi-head attention 的公式:

Multi-Head Attention Formula

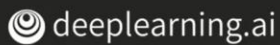
$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, \dots, h_h)W^O$$

where $h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Each head h_i is the attention function of **Query, Key and Value** with trainable parameters (W_i^Q, W_i^K, W_i^V)



reaching different output values.



● 课堂总结

Summary

- Different heads can learn different relationship between words
- Scaled dot-product is adequate for Multi-Head Attention
- Multi-Headed models attend to information from different representations at different positions



different relationships between words from another head.



缩放点积的替代 对于多头注意力至关重要 特别是当矩阵很大时。 多头注意力的主要优点是 它可以并行实现多个查询。

● 当堂测验

试题

The multi-head mechanism calculates attention once.

☒ No

☐ Yes

✓ 正确

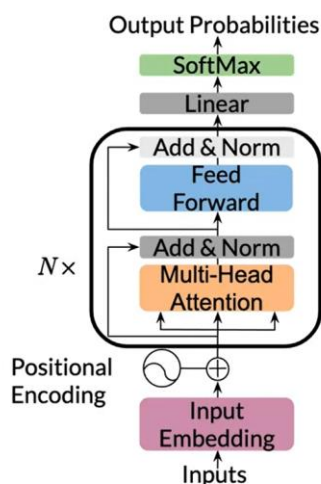
That's right! Runs the attention of the scaled dot-product several times in parallel.

3.6 transformer-decoder

- GPT-2
- transformer-decoder 概述
- (补充) transformer-decoder 基本结构

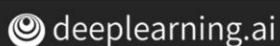
基本架构:

Transformer decoder



Overview

- input: sentence or paragraph
 - we predict the next word
- sentence gets embedded, add positional encoding
 - (vectors representing $\{0, 1, 2, \dots, K\}$)
- multi-head attention looks at previous words
- feed-forward layer with ReLU
 - that's where most parameters are!
- residual connection with layer normalization
- repeat N times
- dense layer and softmax for output



Now don't worry if you
didn't catch it all at once,

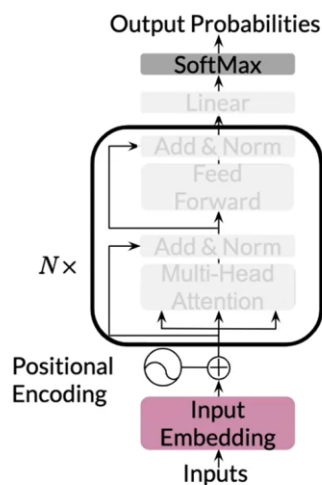
接下来，会将解码器进行逐层介绍：

D_{model} 是嵌入维数，通常是 512, 1024 等 2 的幂次。（现在达到了 10000 多）

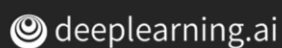
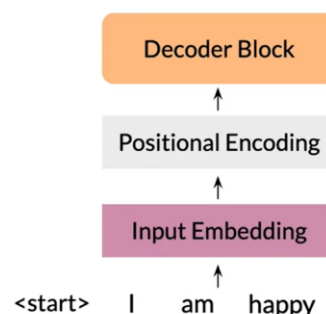
输入向量 ($batch \times length$) $\longrightarrow$ 嵌入向量 ($batch \times length \times D_{model}$) $\longrightarrow$
位置编码 $\longrightarrow N$ 个解码器块

然后是一个完全连接的层，它按长度批量输出形状传感器，通过 vocab 大小。他们记录了 softmax 的交叉熵损失。

Transformer decoder

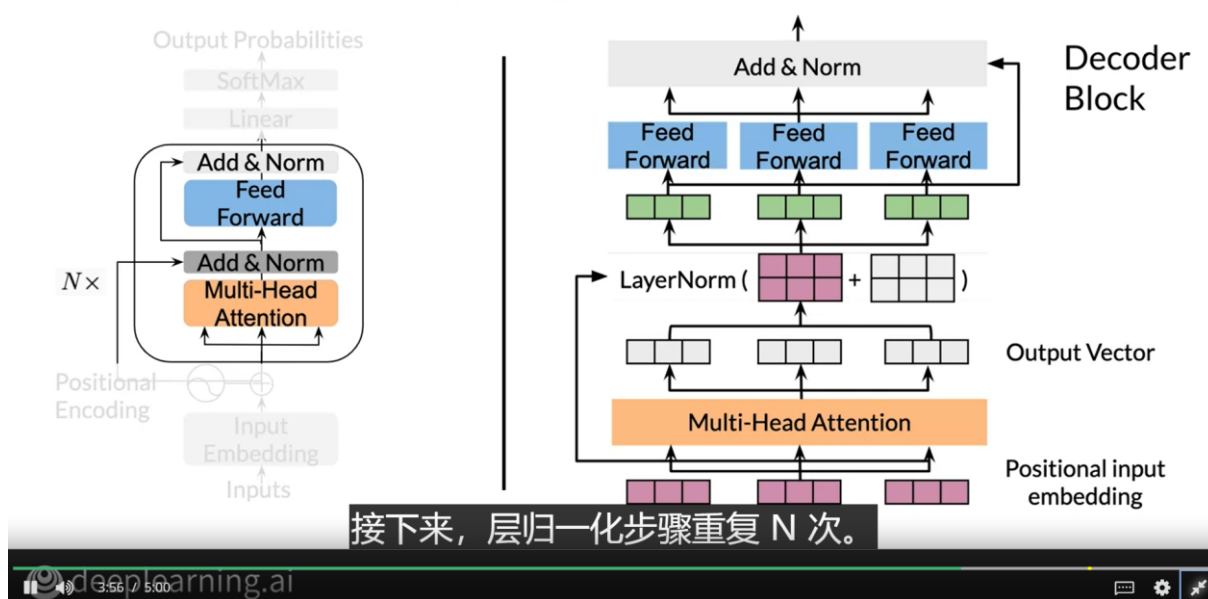


Explanation



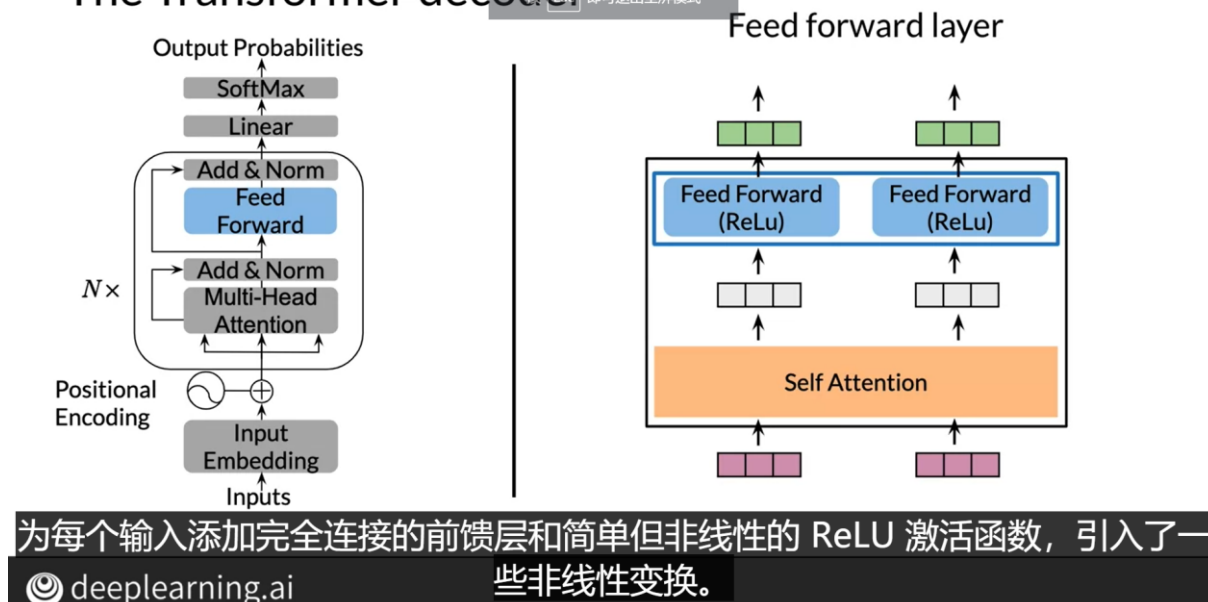
现在让我们看看解码器块是如何构建的。它从这些向量集作为输入序列开始，并添加到 相应的位置编码向量，产生所谓的位置输入嵌入。嵌入后，输入序列将通过多头注意力模型。当此模型处理每个单词，输入序列中的每个位置时，注意本身会搜索序列中的其他位置以帮助识别关系。序列中的每个单词都经过加权。然后，在每一层关注中，周围都有一个残余连接，接下来是图层标准化步骤以加快训练速度，大大减少了整体处理时间。然后，每个单词都将通过前馈层。也就是说，嵌入被馈送到神经网络。最后，您会以正规化形式退出。接下来，将层归一化步骤重复 N 次。最后，获得编码器层输出。

The Transformer decoder



在注意机制和归一化步骤之后，一些非线性通过包括完全连接的前馈来引入转换，每个输入具有简单但非线性的 ReLu 激活功能的三维图层。并且您具有共享的效率参数。前馈神经网络输出向量将本质上替代了原始 RNN 编码器的隐藏状态。

The Transformer decoder



● 课堂总结

Summary

- Transformer decoder mainly consists of three layers
- Decoder and feed-forward blocks are the core of this model code
- It also includes a module to calculate the cross-entropy loss



您看到了用于实现变压器解码器的构建模块。

 deeplearning.ai

● 当堂测验

试题

The decoder has 3 main layers and between them, there is an attention layer that helps the decoder focus on the relevant parts of the input sentence.

- ☒ Yes
- ☐ No

✓ 正确

That's right! That since we do the training on a labeled dataset, we can compare its output with the actual correct output.

3.7 transformer-summarizer

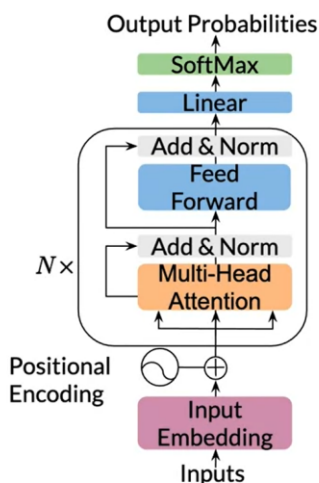
- transformer-summarizer 概述
- 数据预处理的细节
- 使用语言模型进行推论

首先，看一下您将在本周的作业中解决的问题。作为输入，您将获得完整的新闻文章。作为输出，期望您的模型产生文章的摘要。

必须使用加权损失，而不是平均整个序列中每个单词的损失。您用 0 加权文章中单词的损失，而摘要中的那些单词带有 1 的权重。因此，该模型仅关注摘要。

Technical details for data processing

分享



Model Input:

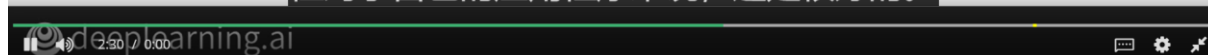
ARTICLE TEXT <EOS> SUMMARY <EOS> <pad> ...

Tokenized version:

[2, 3, 5, 2, 1, 3, 4, 7, 8, 2, 5, 1, 2, 3, 6, 2, 1, 0, 0]

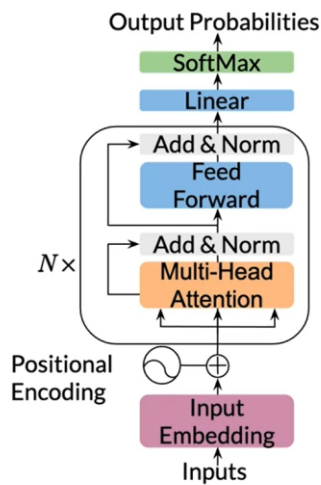
Loss weights: 0s until the first <EOS> and then 1 on the start of the summary.

但对于自己的应用程序来说，这是很好的。



代价函数：交叉熵函数

Cost function



Cross entropy loss

$$J = -\frac{1}{m} \sum_j^m \sum_i^K y_j^i \log \hat{y}_j^i$$

j : over summary

i : batch elements



了要汇总的文章中的单词。

deeplearning.ai

使用语言模型进行推论:

Inference with a Language Model

Model input:

[Article] <EOS> [Summary] <EOS>

Inference:

- Provide: [Article] <EOS>
- Generate summary word-by-word
 - until the final <EOS>
- Pick the next word by random sampling
 - each time you get a different summary!

所以每次运行这个过程，你都会得到一个不同的摘要。

deeplearning.ai

● 课堂总结

Summary

- For summarization, a weighted loss function is optimized
- Transformer Decoder summarizes predicting the next word using
- The transformer uses tokenized versions of the input



以整篇文章作为输入。

 deeplearning.ai

● 当堂测验

试题

The cost function is a cross entropy loss function that ignores the words in the entry to be summarized

- ☒ True
- ☐ False

✓ 正确

That's right! That since we do the training on a labeled dataset, we can compare its output with the actual correct output.

4. 第三周

4.1 本周主要内容

- BERT 双向编码器，使用了 Transformers，如何使用双向性提升性能
- T5 模型：多种输入会有相应的输出。一个模型多种功能。
- 转移学习

BERT: Bi-directional Context

Uni-directional



Bi-directional

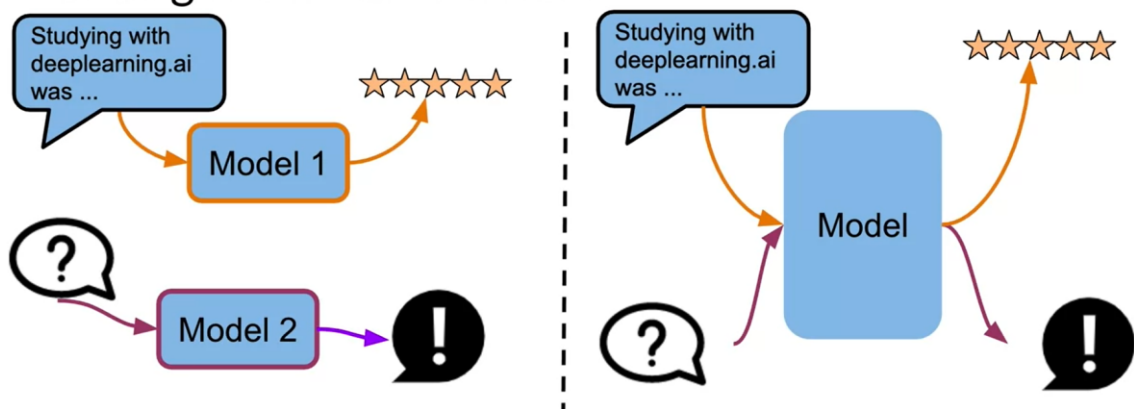


来预测中间词。

deeplearning.ai

- 多任务模型：

T5: Single task vs. Multi task



，而是最终有一个模型。

deeplearning.ai

● 当堂测验

试题

BERT applies Transformer's bi-directional training.

☒ True

☐ False

✓ 正确

That's right! The model that is trained bidirectionally may have a deeper sense of context and language flow than unidirectional models.

● 转移学习的优点

Desirable Goals



- Reduce training time



- Improve predictions



- Small datasets

Transfer Learning!

那么转移学习可能会帮助你。

4.2 NLP 中的转移学习

- 转移学习的目标

- 减少训练时间
- 使用较小的数据集

第一步：

- 转移学习的方法

- 基于功能的方法，词向量
- 微调方法，使用现有的权重并在新任务上进行微调

第二：

- 预训练数据

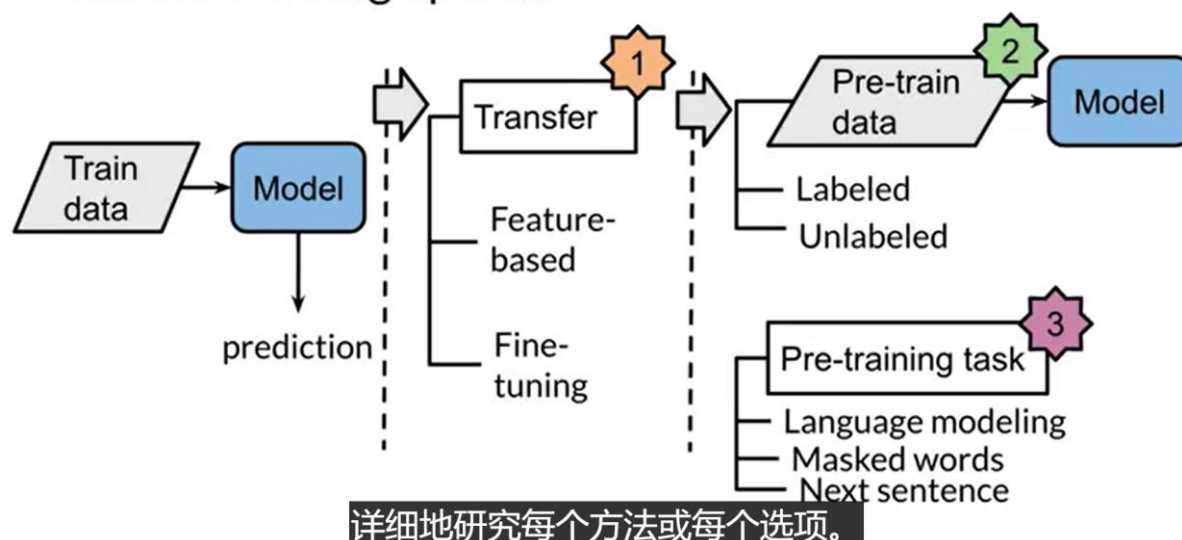
- 已标记的数据
- 未标记的数据

第三步：

- 预训练任务

- 语言模型
- 句子预测
- 单词补充

Transfer learning options

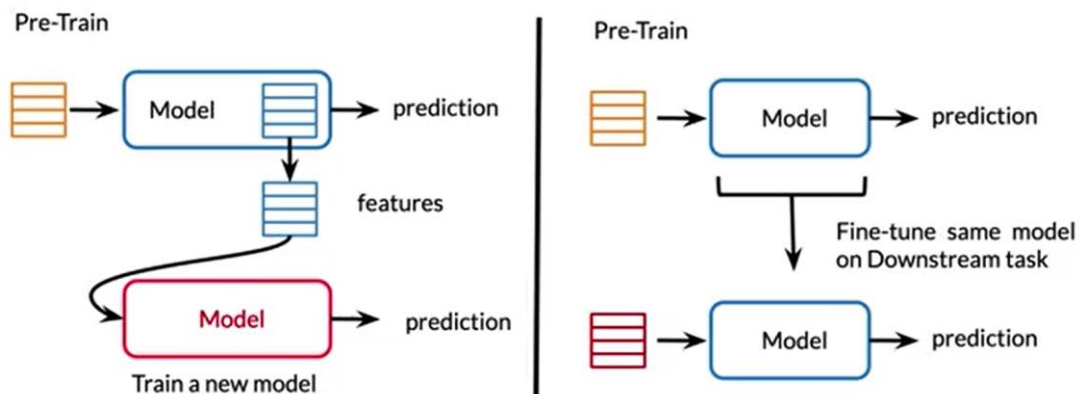


详细地研究每个方法或每个选项。

● 转移学习方法的对比

第一种方法是使用基于特征的学习，它允许你获得高层次的特征。例如，在这里你有一些预先训练。你把它输入一个模型，得到预测，然后用这些权重作为你的词向量。那么这些就成了你的特征。你把它们输入一个新的模型，你就得到了预测。另一方面，对于微调，你有更多的嵌入，你把它输入一个模型，你就得到了一个预测。然后使用这些权重，这些非常相同的权重来微调下游任务上的同一模型。给定在第一个模型中训练的初始权重，首先将它们作为第二个模型的初始化权重。

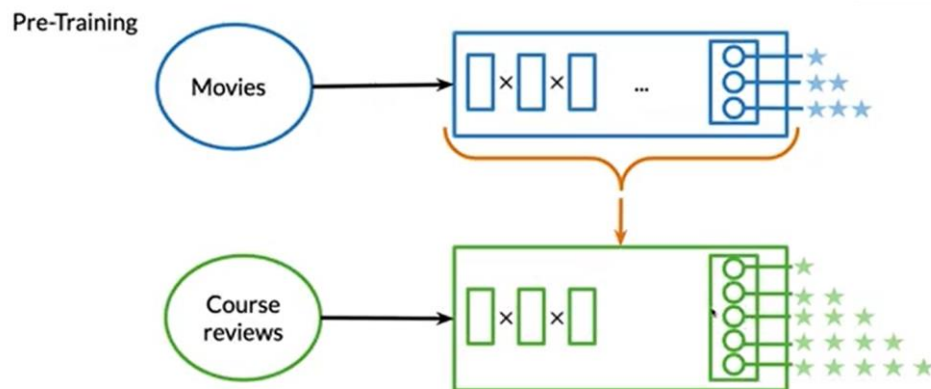
Feature-based vs. Fine-Tuning



第二个模型的初始化权重。

微调还可以增加图层：

Fine-tune: adding a layer



您只能在此输出图层上找到调音，

● 预训练数据集

数据集越大，得到性能好的模型的概率就越大。但是，事实上，我们得到的未标记数据远远大于已标记数据。

Labeled vs Unlabeled Data

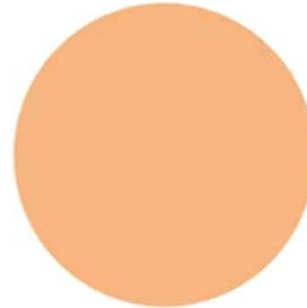
Pre-train
data

2

Labeled text data



Unlabeled text data



未标记的数据比标签数据多。

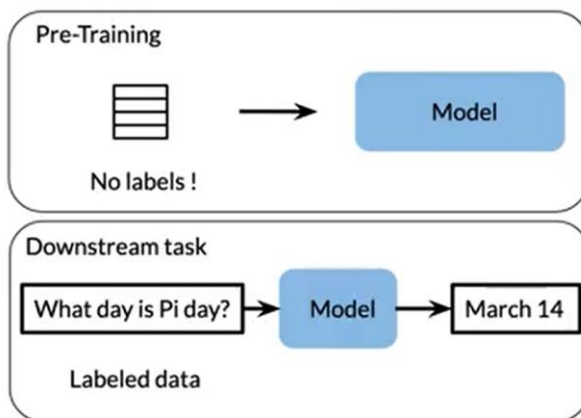
deeplearning.ai

如何使用未标记数据呢？

Transfer learning with unlabeled data

Pre-train
data

2



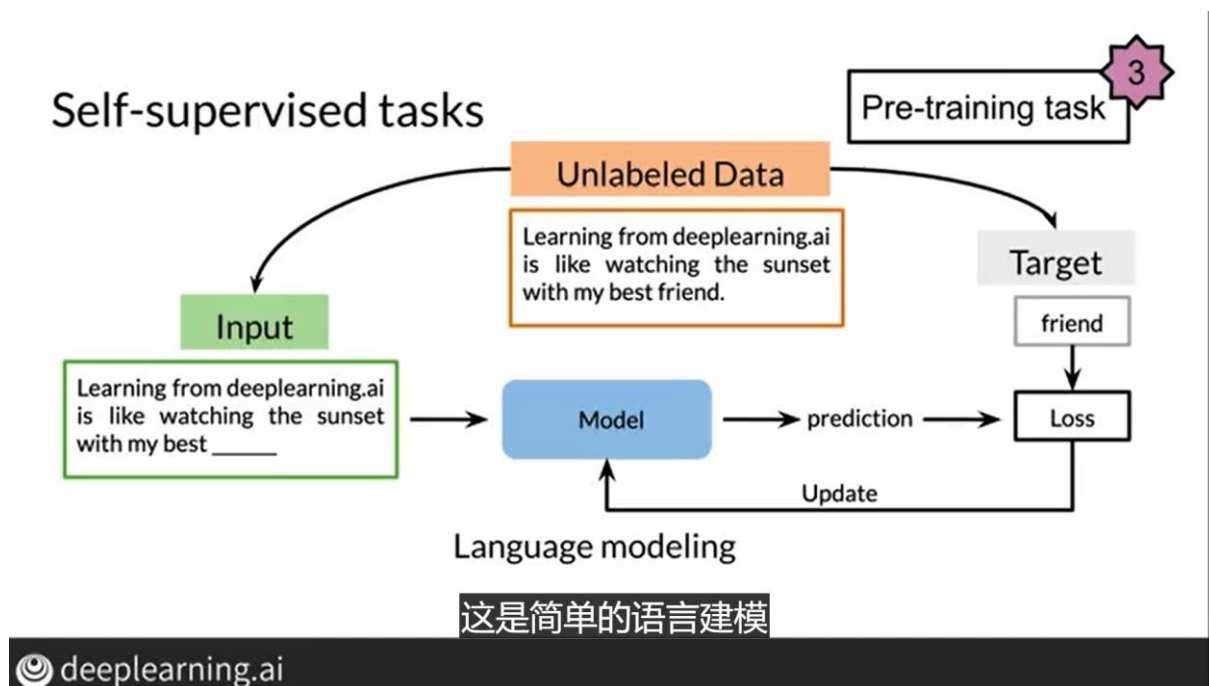
Which tasks work with
unlabeled data?

问题是，哪些任务适用于未标记的数据？

deeplearning.ai

在预训练中，我们可以完全没有标签，例如，我们可以屏蔽随机单词，并尝试预测这些单词是什么，而不需要任何标签。然后在下游任务中，我们可以使用模型权重，并尝试使用标签数据预测，例如，在这里，什么是发薪日，你把它输入到模型中，你的模型将在 3 月 14 日做出响应。问题是，哪些任务使用未标记的数据？自监督任务使用未标记的数据，例如，给定未标记的数据，您可以创建输入或特征。然后可以创建目标或标签。例如，仅仅屏蔽关键字并尝试预测被屏蔽的单词，就是这种方法

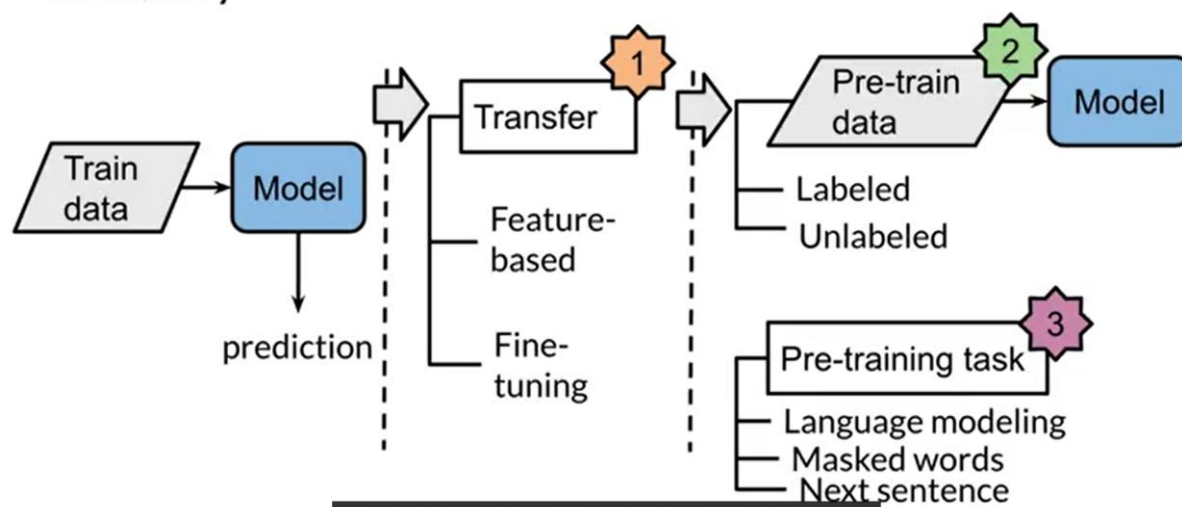
的一个例子。从这些数据来看，没有标记的自我学习深度学习.ai 就像和我最好的朋友一起看日落。你创建了一个输入，你只需屏蔽最后一个单词，朋友。然后你就有了一个模型，这个模型考虑了输入，得到了一个预测。然后你可以看到，目标是朋友，你用预测和目标来获得损失。然后根据你的预测和你得到的损失来更新模型。这是一个简单的语言建模，您现在应该已经熟悉了。



为每个下游任务微调一个模型，首先，你有一些预先训练过的模型，然后你使用同样的模型，比如说，把英语翻译成德语，或者做总结，或者回答问题。你使用初始权重，你只是微调不同的任务。总之，你已经看到了一些数据，你把它输入你的模型，你就得到了一个预测。但是现在，通过迁移学习，您可以使用基于特征的方法，如单词向量或微调方法。这将引导您使用某种类型的预训练数据，用于标记和未标记的数据集，或者使用预训练任务（如 mask）进行建模或下一句预测。你现在已经看到了转移学习的一些优点。

● 当堂总结

Summary



如掩码) 来进行建模或下一句预测。

● 当堂测验

试题

Why transfer learning in NLP?

☒ Many NLP tasks share a common knowledge about language.

✓ 正确

☐ It requires a larger dataset and more training time.

☒ Tasks can inform each other.

✓ 正确

☒ It is better to use a previously trained model as a starting point for solving a problem rather than building a model from scratch.

✓ 正确

4.3 ELMo、GPT、BERT、T5

- 模型的发展历程

Outline



请注意，这并不是所有相关模型和研究结果的完整历史，但它有助

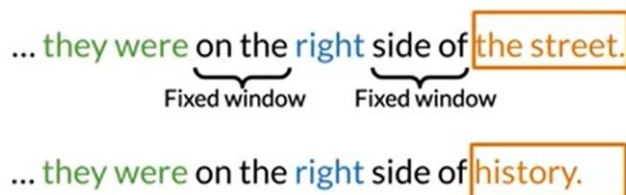
deeplearning.ai

- CBOW

使用固定大小为 C 的窗口，预测中心词。 C 是固定的，一旦需要用到全文信息，这个方法就太过僵硬，不够灵活。

分享

Need more context?



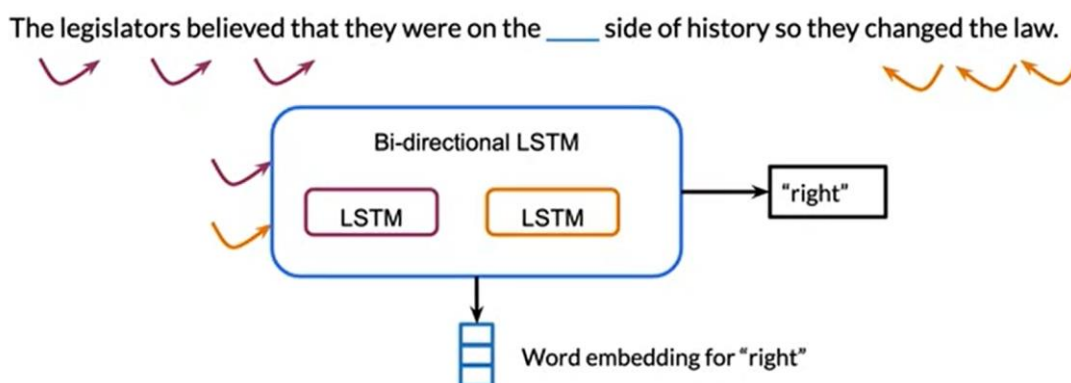
这可能是历史。

deeplearning.ai

- ELMo

对待预测词的左右两侧，分别使用 RNN（或双向 LSTM），进行预测。

ELMo: Full context using RNN

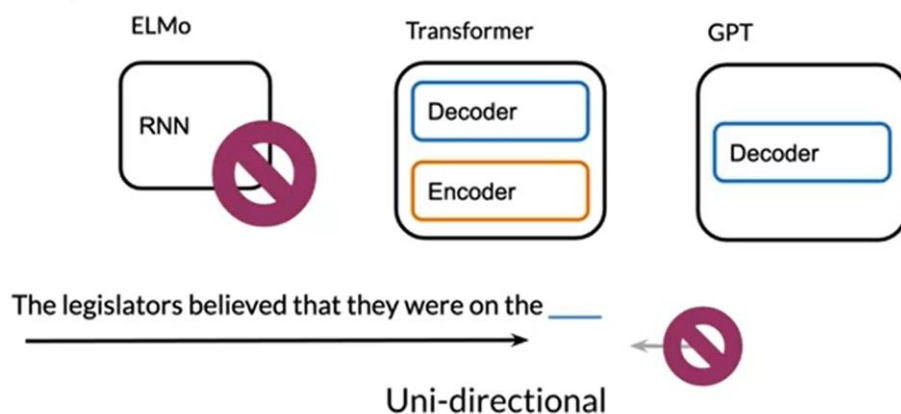


从这个模型中获得单词的嵌入。

deeplearning.ai

- open AI 带来了 GPT

Open AI GPT



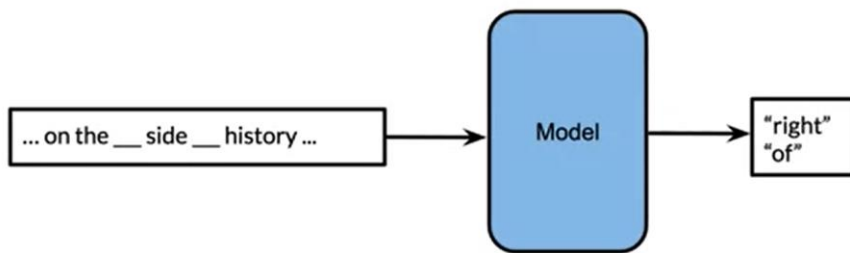
另一边的东西，而不幸的是 GPT 是单向的。

deeplearning.ai

为什么不使用双向呢？

在变压器中，请记住我们有以下几点，这是自我关注，每个词都能窥视自己。但是这里的问题是 GPT 仍然是单向的。记住以前的课程，您已经看到了因果关系关注以及如何只看前面的输入，所以没有偷看。回顾一下，我们有转换器，有编码器和解码器，我们的 GPT 只是一个解码器，然后我们有了 BERT，这只是一个编码器。

Transformer + Bi-directional Context



Multi-Mask Language Modeling

你得到正确的输出。



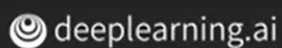
- BERT

BERT: Words to Sentences



Next Sentence Prediction

哪一句是句子 B。

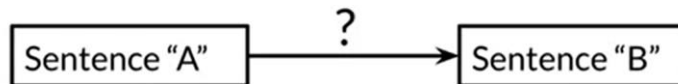


BERT Pre-training Tasks

Multi-Mask Language Modeling



Next Sentence Prediction

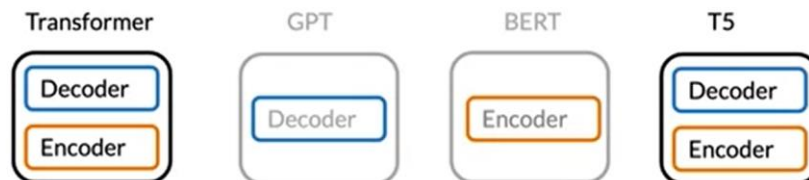


你预测哪一个是句子 B



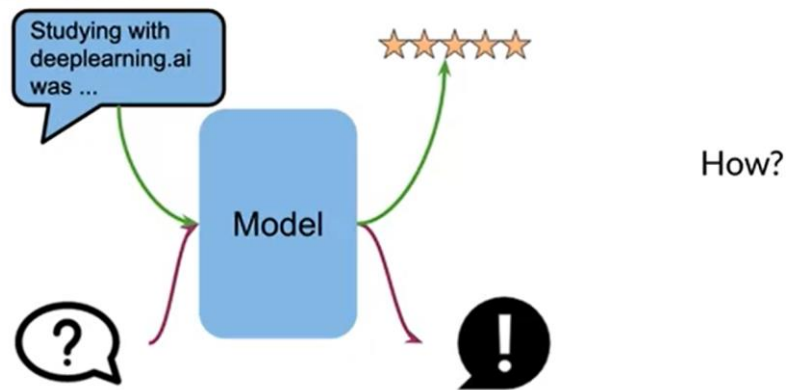
- T5

T5: Encoder vs. Encoder-Decoder



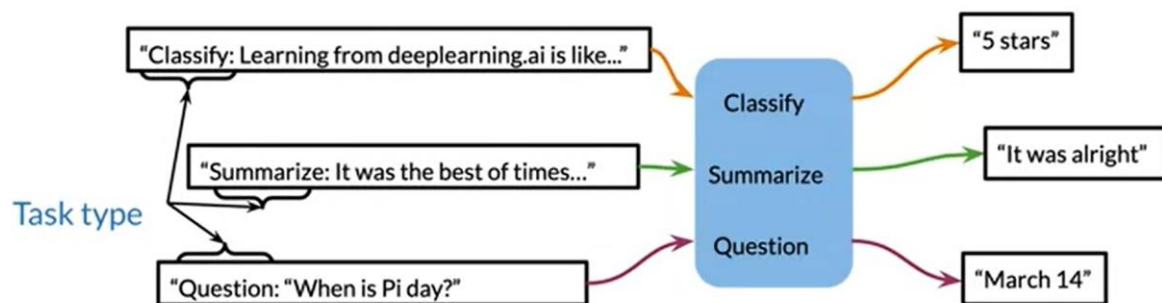
一下，我们有使用编码器解码器的变压器，我们有 GPT，它只使用解码器，我们
有 BERT 只使用编码器，然后我们得到了 T5，它使用编码器和解码器。

T5: Multi-task



怎么能做到这一点?

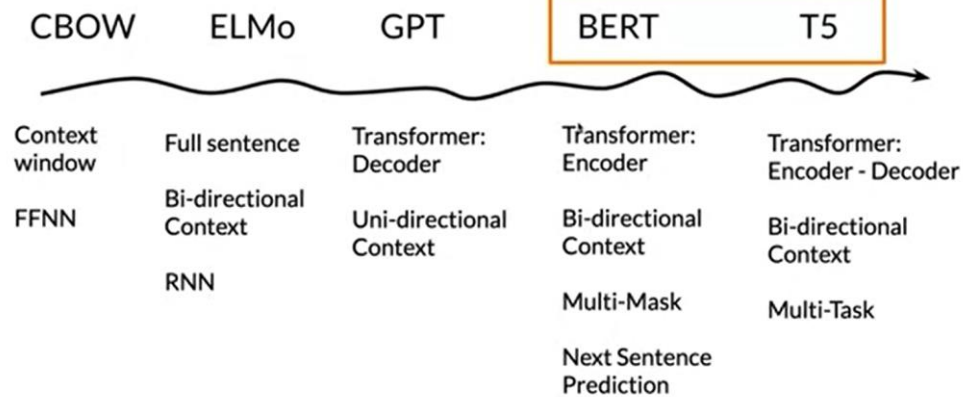
T5: Text-to-Text



一个标签，然后模型将处理输出。

- 当堂总结

Summary



然后我们谈到 BERT 以及它是如何只是一个编码器。

deeplearning.ai

● 当堂测验

试题

ELMo is unidirectional?

- ☒ False
☐ True

✓ 正确

That's right! It uses a task-specific trained bidirectional LSTM to create embeddings.

4.4 BERT

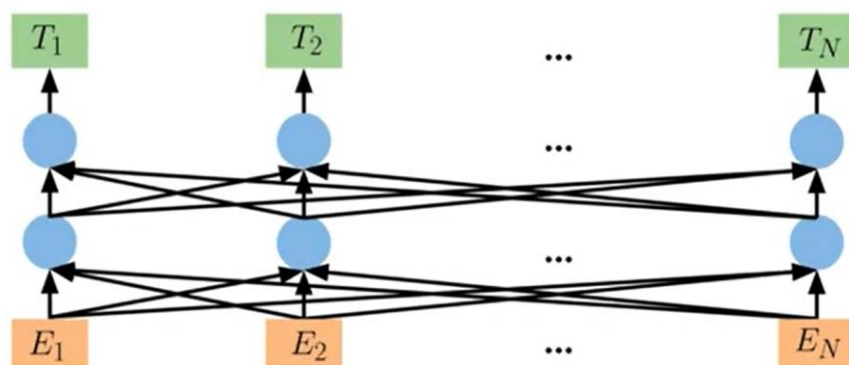
Bidirectional Encoder Representations from Transformers (BERT)

- BERT 框架

在预训练期间，在不同的预训练任务中使用未标记的数据对模型进行训练。如您所见，为了进行微调，BERT 模型使用的预训练的参数和所有参数都是来自下游任务的标记数据进行微调。例如，在上面的图中，您将获得相应的嵌入。您经历了几个变压器块，然后，您进行预测。

BERT

- Makes use of transfer learning/pre-training:



BERT 的框架中有两个步骤，即预训练和微调。

BERT

- A multi layer bidirectional transformer

- Positional embeddings

- BERT_base:

12 layers (12 transformer blocks)

12 attentions heads

110 million parameters

12 个注意头和 1.1 亿个参数，而



● 预训练

在将单词序列输入到 BERT 模型之前，我们先屏蔽 15%的单词，然后训练数据生成器随机选择其中 15%的位置进行预测。然后，如果选择了 ice 令牌，我们用一个替换 ice 令牌，80%的时间替换掩码令牌，然后，2 个，10%的随机令牌，然后，3 个，10%的时间不变的 ice 令牌。在本例中，您在上一张幻灯片中看到的 TI 将用于预测具有交叉熵损失的原始令牌。在本例中，这被称为掩码语言模型。在这里，我们有“放学后卢卡斯在图书馆做他的空白”，也许是工作，也许是家庭作业。其中一个词，你的伯特模型将试图预测。为此，通常，您所做的，只需在 TI 令牌之后添加密集层，并在编码器输出后使用它进行分类。你只需将输出的向量乘以嵌入矩阵，然后，把它们转换成词汇表维度，然后在末尾添加一个 softmax。这是另一句话，放学后卢卡斯在图书馆做作业。然后，放学后把他的作业空白了。你必须预测卢卡斯的所作所为，然后，图书馆。总之，您可以随机选择 15%的令牌。你在 80%的时间里屏蔽，10%的时间用随机标记替换，或者 10%的时间保持原样，然后，知道一个句子中可能有多个被屏蔽的跨度。你可以在同一个句子中匹配多个单词，在 BERT 中，下一个句子预测也在预训练时使用。给出两句话，如果是真的，那就意味着这两个句子是一个接一个的。

BERT pre-training

After school Lukasz does his _____ in the library.

- Masked language modeling (MLM)



- 当堂总结

Summary

- Choose 15% of the tokens at random: mask them 80% of the time, replace them with a random token 10% of the time, or keep as is 10% of the time.
- There could be multiple masked spans in a sentence
- Next sentence prediction is also used when pre-training.



- 当堂测验

试题

BERT is designed to pretrain deep bi-directional representations from unlabeled text.

- ☒ Yes
☐ No

✓ 正确

That's right! BERT is previously trained in different, but related, masking language modeling and predicting the next sentence.

4.5 BERT 目标

本节你将会看到，尝试最大限度的减少什么。

● 本节大纲

Outline

- Understand how BERT inputs are fed into the model
- Visualize the output
- Learn about the BERT objective

然后你要可视化输出。

 deeplearning.ai

● BERT 的输入

Formalizing the input

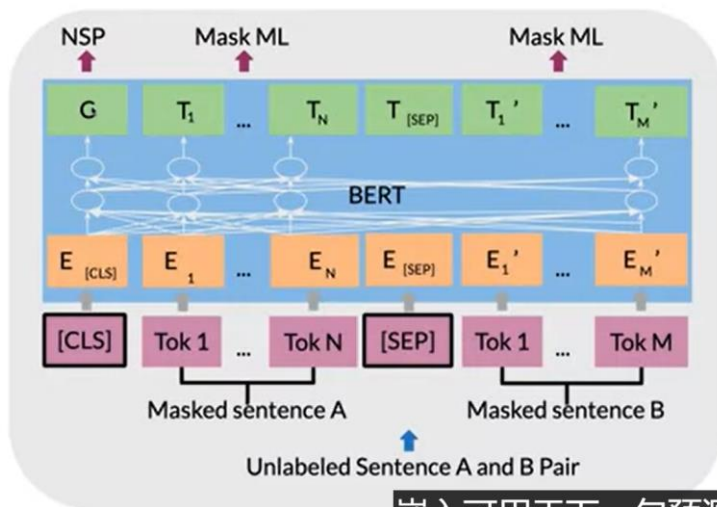
Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{\# \# ing}$	$E_{[SEP]}$
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

然后你得到你的新输入。

deeplearning.ai

- 可视化输出

Visualizing the output



- **[CLS]**: a special classification symbol added in front of every input
- **[SEP]**: a special separator token

嵌入可用于下一句预测。

deeplearning.ai

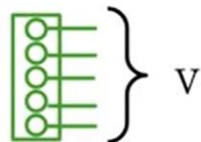
- BERT 的目标

不同任务目标不同

BERT Objective

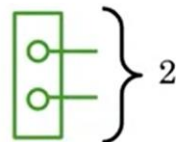
Objective 1:
Multi-Mask LM

Loss: Cross Entropy Loss



Objective 2:
Next Sentence Prediction

Loss: Binary Loss



鉴于这两个句子跟随彼此或不。

 deeplearning.ai

● 当堂测验

试题

BERT is an open source machine learning framework for natural language processing?

☒ True

☐ False

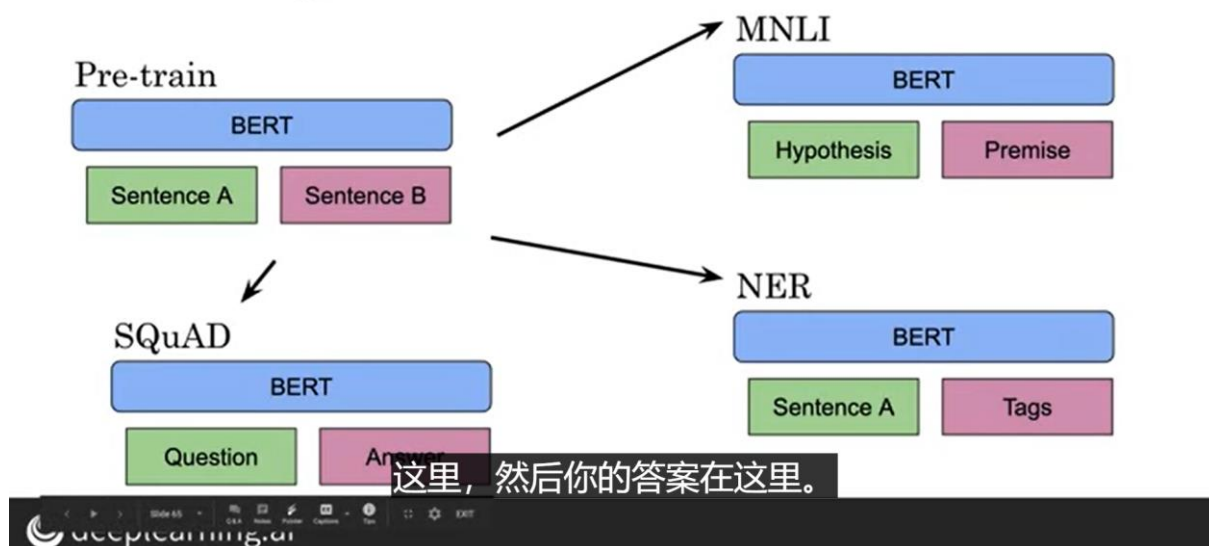
✓ 正确

That's right! BERT is designed to help understand the meaning of ambiguous language in text to establish context.

4.6 BERT 的微调

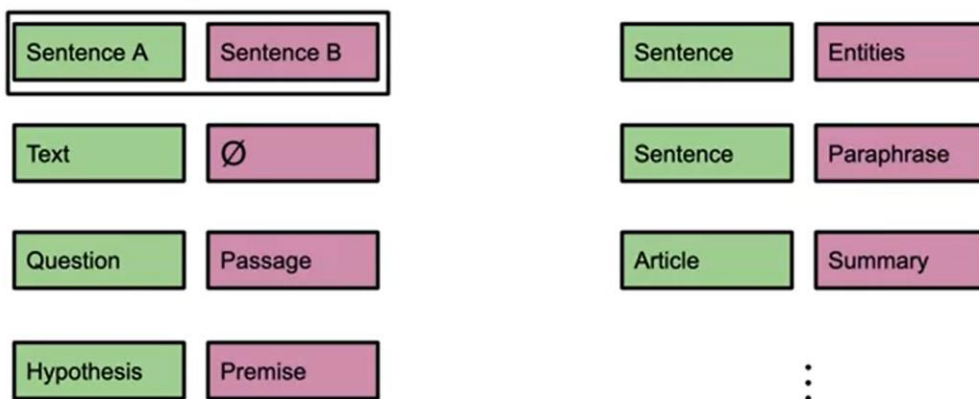
本节介绍 BERT 的微调，这样就可以在自己的数据集上使用它。

Fine-tuning BERT: Outline



- 当堂总结

Summary



这只是 BERT 模型的输入。

- 当堂测验

试题

What are the advantages of Fine-tuning?

☒ Much less time is needed to train the fitted model

✓ 正确

☐ A model must be trained from scratch a large dataset with reasonable accuracy, spending a lot of time and energy creating the dataset.

☒ Allows to adjust the task in a much smaller dataset.

✓ 正确

☒ Achieves good results with minimal task-specific adjustments for a wide variety of tasks.

✓ 正确

4.7 T5 Transformer

T5 可以用于多个 NLP 任务。具体地讲，它使用了迁移学习和掩盖语言模型。

● 课堂大纲

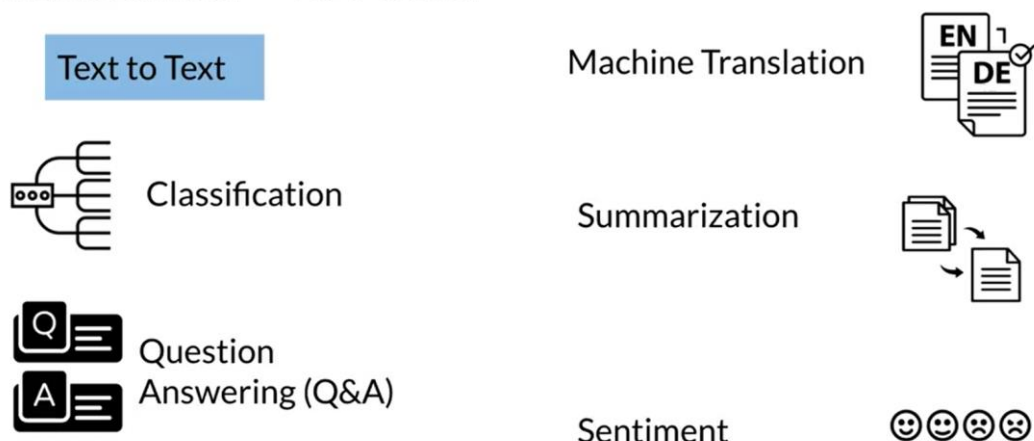
Outline

- Understand how T5 works
- Recognize the different types of attention used
- Overview of model architecture

以及模型架构的团队概述。

● T5 的用途

Transformer - T5 Model



©Exploring the Limits of Transfer learning w

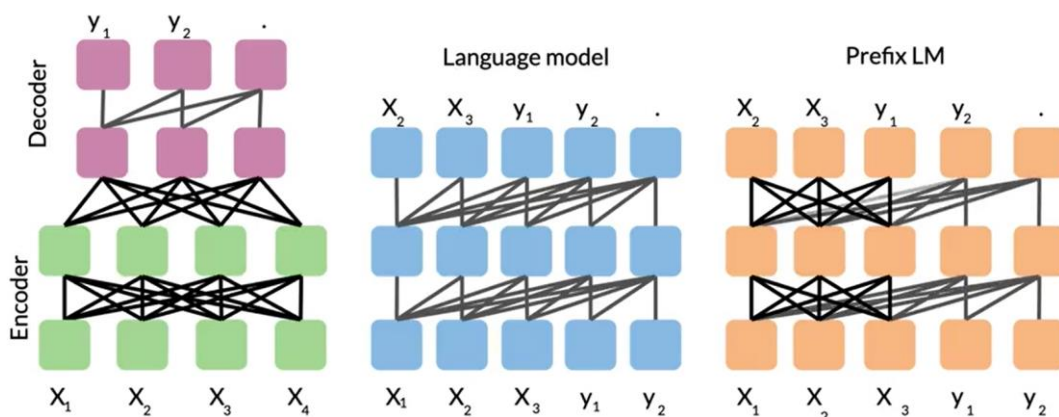
现在我们将专注于这四个应用程序。

 deeplearning.ai

● 模型架构

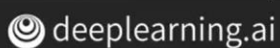
因此，我们从基本的编码器-解码器表示开始。因此，您可以在此处看到编码器和然后在解码器中引起因果关系。然后，您将拥有通用的编码器-解码器表示形式，就像符号。因此，浅灰色线对应于因果掩盖。深灰色线对应于完全可见的蒙版。因此，正如我再说一遍，这是标准的编码器-解码器体系结构。在这里的中间，我们有由单个转换器层堆栈组成的语言模型。它正在被馈送输入和目标的串联。因此，如您所见，它始终使用因果屏蔽，因为它们是所有灰线。X1 进入了这里，到达 X2，X2 进入模型 X3，依此类推。现在在这里的右边，我们有前缀语言模型，它对应于完全允许您可以在该处的黑色箭头中看到对输入的可见遮罩。然后在其余部分进行因果掩盖。

Model Architecture



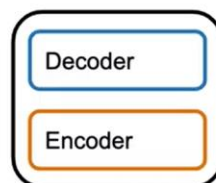
©Exploring the Limits of Transfer Learning

所以你可以在这里看到，它在做因果掩盖。



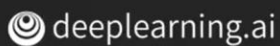
Model Architecture

- Encoder/decoder
- 12 transformer blocks each
- 220 million parameters



©Exploring the Limits of Transfer Learning

所以你可以想象它是一个十几个鸡蛋，然后是 2.2 亿个参数。



- 课堂总结

Summary

- Prefix LM attention
- Model architecture
- Pre-training T5 (MLM)

我们只是在这里使用掩模语言建模。



● 当堂测验

试题

T5 is an encoder-decoder model previously trained in a multi-tasking mix.

- ☒ True
- ☐ False

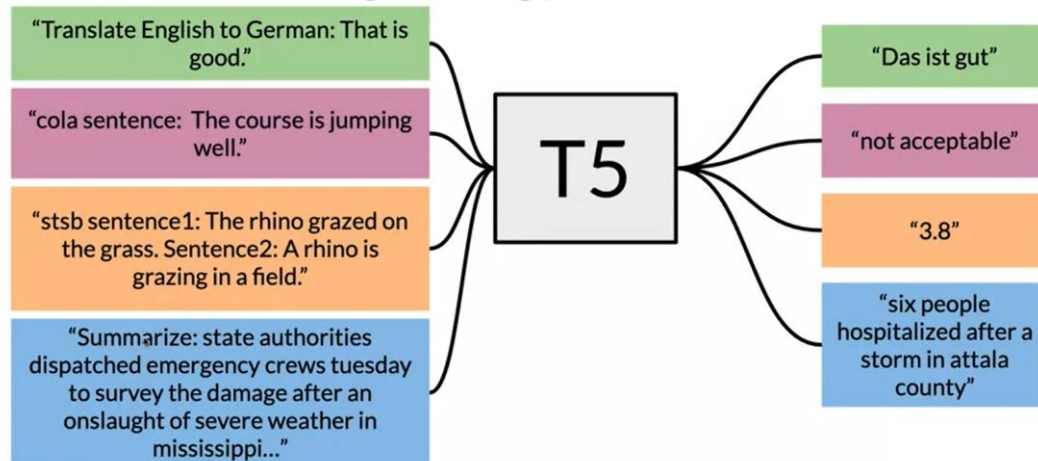
✓ 正确

That's right! Each task is converted into a text-to-text format.

4.8 多任务训练策略

您将看到如何训练一个模型使您在几个 NLP 任务上获得非常好的结果。

Multi-task training strategy



©Exploring the Limits of Transfer learning with a unified text-to-text architecture

This is how it works.



deeplearning.ai

● 输入和输出形式

Input and Output Format

Machine translation:

- translate English to German: That is good.
- Predict entailment, contradiction , or neutral
- mnli premise: I hate pigeons hypothesis: My feelings towards pigeons are filled with animosity. target: entailment
- Winograd schema
- The city councilmen refused the demonstrators a permit because *they* feared violence

©Exploring the Limits of Transfer learning with a unified text-to-text architecture

被任务预测 “他们” 作为市议员。



deeplearning.ai

● 重点关注 GLUE 评分

Multi-task Training Strategy

Fine-tuning method	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
* All parameters	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Adapter layers, $d = 32$	80.52	15.08	79.32	60.40	13.84	17.88	15.54
Adapter layers, $d = 128$	81.51	16.62	79.47	63.03	19.83	27.50	22.63
Adapter layers, $d = 512$	81.54	17.78	79.18	64.30	23.45	33.98	25.81
Adapter layers, $d = 2048$	81.51	16.62	79.47	63.03	19.83	27.50	22.63
Gradual unfreezing	82.50	18.95	79.17	70.79	26.71	39.02	26.93

How much data from each task to train on?

©Exploring the Limits of Transfer learning

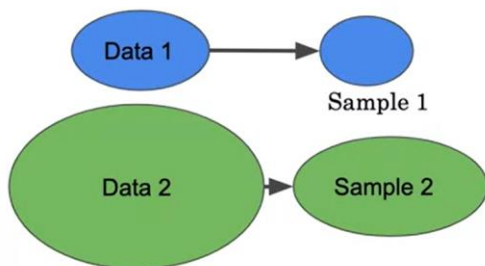
在许多任务中实际上达到了最先进的水平。

deeplearning.ai

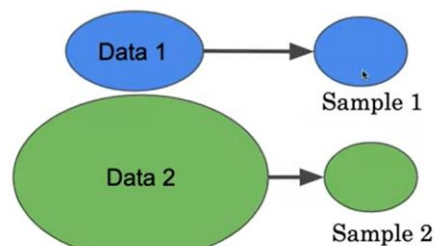
- 数据选取策略
 - 按比例选取
 - 按固定数量选取
 - 上面两种方法的折中

Data Training Strategies

Examples-proportional mixing



Equal mixing



Temperature-scaled mixing

中间有一些叫做温度调节的混合，在

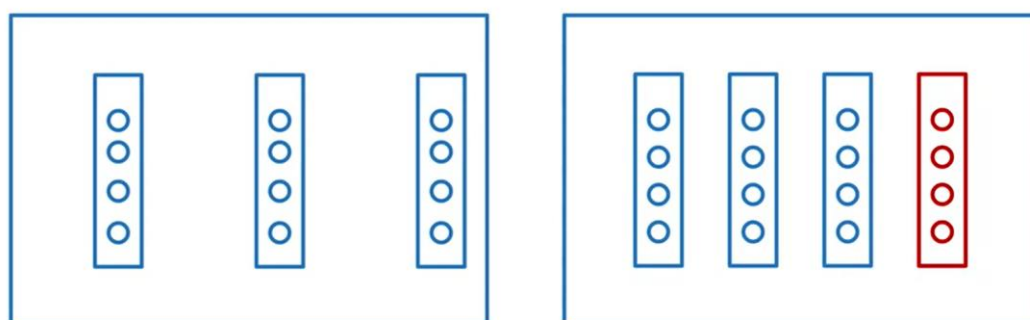
deeplearning.ai

- 逐步解冻与适配器层

在逐渐解冻的过程中，最终会发生什么，您一次冻结一层。你说这是你的神经网络

网络你冻结了最后一个，你使用微调，使其他固定。然后冻结这一个然后冻结这一个因此您将逐步解冻每一层。对于适配器层，您基本上可以添加现有的神经网络，或者向中添加神经网络 每个前馈和变压器的每个块。然后这些新的前馈网络，他们的设计是为了输出尺寸与输入匹配。这样可以将它们插入没有任何结构上的变化。微调时，仅这些新适配器层和层归一化参数正在更新。

Gradual unfreezing vs. Adapter layers



Gradual unfreezing

Adapter layers

©Exploring the Limits of Transfer learning

在没有任何结构改变的情况下插入它们。



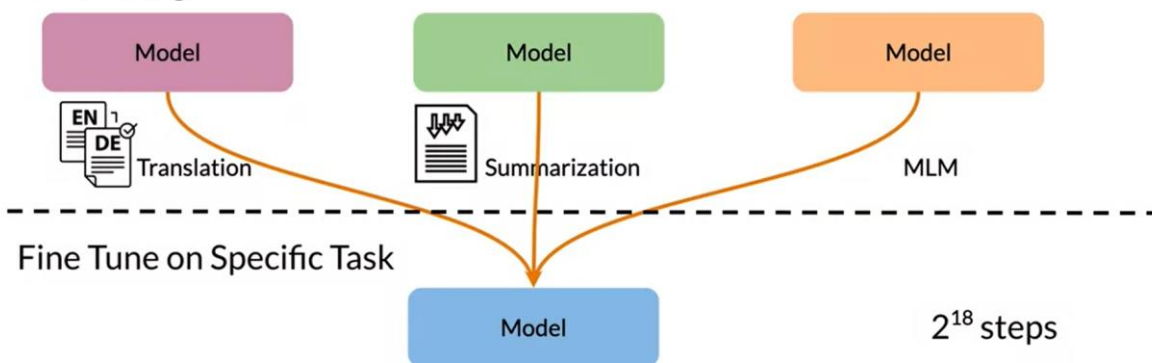
deeplearning.ai

● 微调

现在，我们将详细讨论微调。通常采用的方法 这里使用的目的是训练 一个可以同时执行多个任务。例如模型 它的大部分参数 在所有任务之间共享。我们可能会在很多任务上训练一个模型，但是在报告效果时，我们可以为每个任务选择不同的检查点。在这里，任务可能是翻译，摘要或掩盖的语言建模。

Fine-tuning

Pre Training



同时执行多项任务的单个模型。

4.9 GLUE 基准

- GLUE 解释（最后一个排行榜）

General Language Understanding Evaluation

- A collection used to train, evaluate, analyze natural language understanding systems
- Datasets with different genres, and of different sizes and difficulties
- Leaderboard

们可以使用数据集，看看他们的模型与其他模型相比有多好。

 deeplearning.ai

- GLUE 的使用场景

Tasks Evaluated on

- Sentence grammatical or not?
- Sentiment
- Paraphrase
- Similarity
- Questions duplicates
- Answerable
- Contradiction
- Entailment
- Winograd (co-ref)



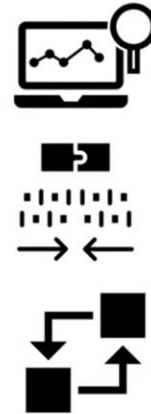
代词是指某个名词还是另一个名词。

 deeplearning.ai

- GLUE 优点

General Language Understanding Evaluation

- Drive research
- Model agnostic
- Makes use of transfer learning



最后，它允许您利用转移学习，



● 当堂检测

GLUE is a collection of resources for training, evaluating and analyzing natural language understanding systems.

- ☒ True
- ☐ False

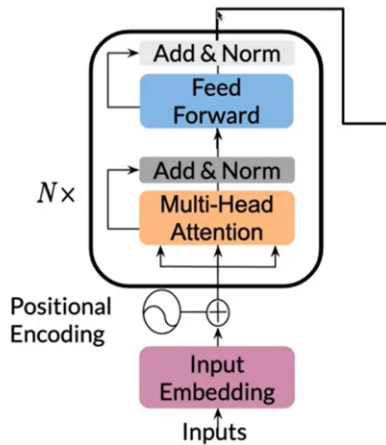
✓ 正确

That's right! It consists of a benchmark of nine different language comprehension tasks, an ancillary data set, and a platform for evaluating and comparing the models.

4.10 问答 QA

- 前馈层块

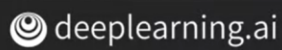
Transformer encoder



Feedforward:

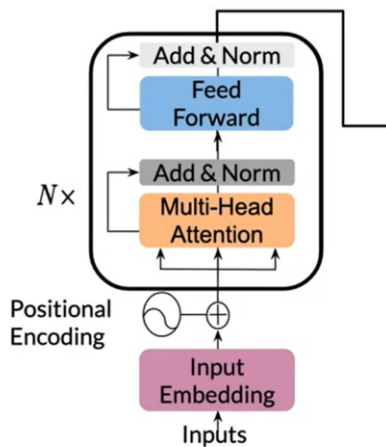
```
[  
    LayerNorm,  
    dense,  
    activation,  
    dropout_middle,  
    dense,  
    dropout_final  
]
```

规范化，然后进入解码器体系结构。



- 编码器块

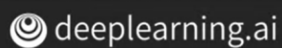
Transformer encoder



Encoder block:

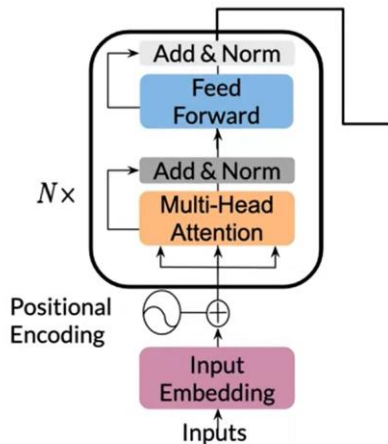
```
[  
    Residual(  
        LayerNorm,  
        attention,  
        dropout_,  
    ),  
    Residual(  
        feed_forward,  
    ),  
]
```

然后它在前进中有另一个残余连接。



- 放一块看

Transformer encoder



Feedforward:

```
[
    LayerNorm,
    dense,
    activation,
    dropout_middle,
    dense,
    dropout_final
]
```

Encoder block:

```
[
    Residual(
        LayerNorm,
        attention,
        dropout_,
    ),
    Residual(
        feed_forward,
    )
]
```

所以把它们放在一起，你可以在这里前进，然后你有

deeplearning.ai

● QA 示例

分享

Data examples

Question: What percentage of the French population today is non - European ?

Context: Since the end of the Second World War , France has become an ethnically diverse country . Today , **approximately five percent** of the French population is non - European and non - white . This does not approach the number of non - white citizens in the United States (roughly 28 - 37 % , depending on how Latinos are classified ; see Demographics of the United States) . Nevertheless , it amounts to at least three million people , and has forced the issues of ethnic diversity onto the French policy agenda . France has developed an approach to dealing with ethnic problems that stands in contrast to that of many advanced , industrialized countries . Unlike the United States , Britain , or even the Netherlands , France maintains a " color - blind " model of public policy . This means that it targets virtually no policies directly at racial or ethnic groups . Instead , it uses geographic or class criteria to address issues of social inequalities . It has , however , developed an extensive anti - racist policy repertoire since the early 1970s . Until recently , French policies focused primarily on issues of hate speech — going much further than their American counterparts — and relatively less on issues of discrimination in jobs , housing , and in provision of goods and services .

Target: **Approximately five percent**

所以答案是大约 5%，这可以在这里找到。

● T5 构建问答系统

Implementing Q&A with T5

- Load a pre-trained model
- Process data to get the required inputs and outputs: "question: Q context: C" as input and "A" as target
- Fine tune your model on the new task and input
- Predict using your own model

联系人：后面是上下文作为输入，A 作为答案的目标。



● 当堂检测

试题

T5 is a text-to-text shared frame where the input and output are always strings?

- ☒ Yes
- ☐ No

✓ 正确

That's right! Is a text-to-text framework that receives some context along with a question and is able to find the answer to the question in the context.

5. 第四周

5.1 对话机器人简介

Dragon 模型基于 GPT-3。它基于所有先前的回合作为输入来生成交互式故事。这使得使用非常长序列的任务成为可能。看看这个！

<https://play.aidungeon.io/main/landing>

分享

Long Text Sequences

Tasks In NLP:



Writing Books



Chatbots

许多基于 GPT-3 变压器模型，



处理长文本序列是构建聊天机器人的核心。聊天机器人模型需要使用之前的所有对话内容作为输入下一个答复，这可以使某些非常大的上下文成为 Windows。

5.2 Transformer 复杂度

注意长度为 L 的序列，需要 L 平方的时间和内存。例如，如果您正在注意长度为 L 的两个句子，您需要将第一句话中的每个单词与第二句话中的每个单词作比较，因此时间和内存的复杂度是 L 乘以 L 比较或 L 平方。如果您有 N 层注意力，那么它需要 N 倍的内存。

● Attention 复杂度

Attention Complexity

- Attention: $\text{softmax}(QK^T)V$
- Q, K, V are all $[L, d_{\text{model}}]$
- QK^T is $[L, L]$

所以 Q 乘以 K 转置将是 L 由 L 或 L 平方。

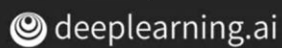
 deeplearning.ai

● N 层的内存

Memory with N Layers

- Activations need to be stored for backprop
- Big models are getting bigger
- Compute vs memory tradeoff

更少的内存是有效的，接下来我会告诉你如何做到这一点。

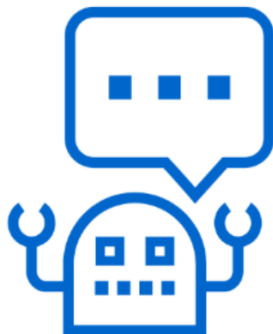


● 当堂测验

试题

The more layers a Transformer has, the more memory it needs in training. This is because you need to save the forward pass activations for backprop.

What is a way to overcome this memory requirement?



- ☐ Train in the cloud.
- ☒ Recompute activations.

跳过

继续

5.3 LSH Attention

- Attention 做了什么？

What does Attention do?

Select Nearest Neighbors (K,Q) and return corresponding V

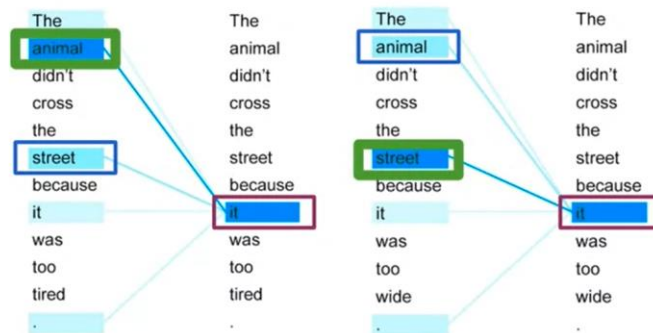


image ©
([Transformer: A Novel Neural Network Architecture for Language Understanding.](#))

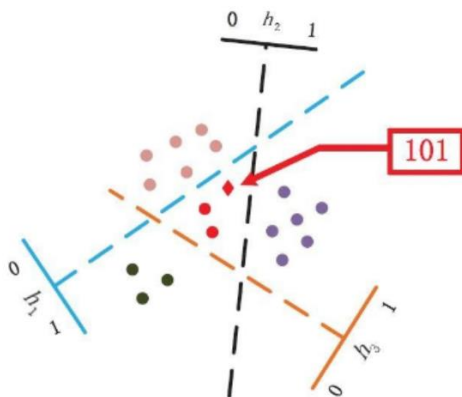
在这两个注意事项中，你可以看到它是动物或街道。

deeplearning.ai

- 当堂测验

试题

Locality sensitive hashing reduces the computational cost of finding k nearest neighbors.



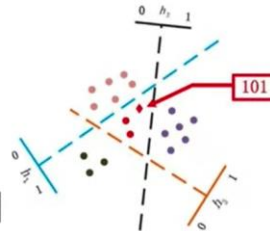
- ☐ False.
- ☒ True.

- LSH

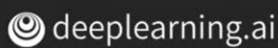
Nearest Neighbors

Compute the nearest neighbor to q among vectors $\{k_1, \dots, k_n\}$

- Attention computes $d(q, k_i)$ for i from 1 to n which can be slow
- Faster *approximate* uses locality sensitive hashing (LSH)
- Locality sensitive: if q is close to k_i :
 $\text{hash}(q) == \text{hash}(k_i)$
- Achieve by randomly cutting space
 $\text{hash}(x) = \text{sign}(xR)$ $R: [d, n_hash_bins]$



随机的，维度的大小为 d 乘以哈希条柱的数量。



● 解释一下利用 LSH 注意力来加快传统的注意力模型：

首先，您对 Q 和 K 进行哈希处理，然后执行标准注意，这是在相同的哈希条柱中。这样可以将每个 K 的搜索空间减少到与 Q 相同的 LSH 存储桶。您可以多次重复此过程以增加在同一单元格中找到 Q 和 K 的可能性，并且可以有效地利用并行计算。

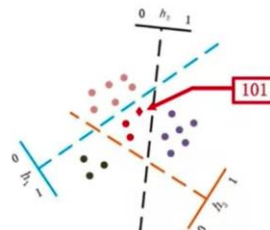
LSH Attention

Standard Attention:

$$A(Q, K, V) = \text{softmax}(QK^T)V$$

LSH Attention:

- Hash Q and K
- Standard attention within same-hash bins
- Repeat a few times to increase probability of key in the same bin



以增加在同一条柱中找到 Q 和 K 的概率。



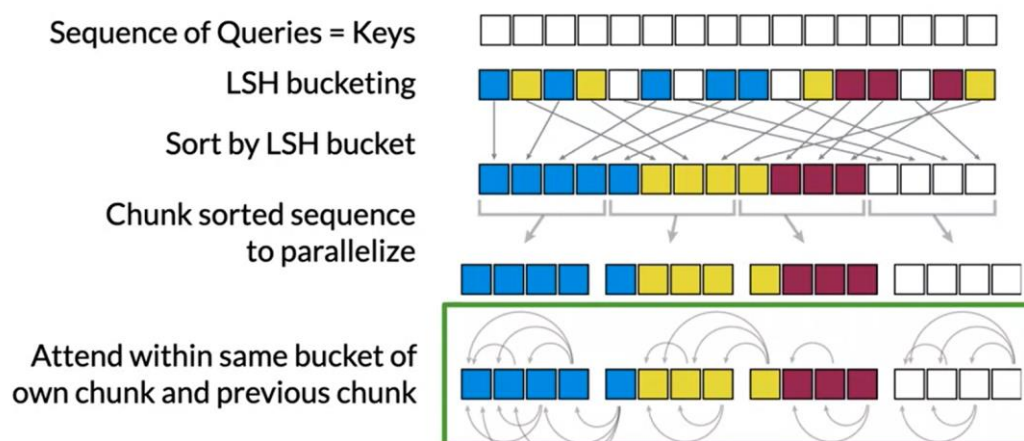
● 如何将 LSH 集成到注意层中：

首先，修改模型，使其在每个位置输出一个矢量，它既用作查询又用作键。这称为 QK 注意，其性能与常规注意一样好。接下来，您可以使用 LSH 将向量放置到存储桶

中。然后按 LSH 存储桶对向量进行排序，然后，最后，您只在每个桶中进行关注。您一次可以做一个水桶，但是没有利用硬件并行性。相反，我将向您展示如何进行批处理计算。第一步，批处理是将排序的序列分成固定大小的块。这样可以进行一些并行计算，然后您让每个块都参与其中。相邻的大块通过散列的哈希桶发现情况不止一块，就像您在这里看到的蓝色，黄色和洋红色水桶一样。这就是 LSH 关注的核心。最后要考虑的一点是 LSH 是一种概率模型，而不是确定性模型。这是因为 LSH 算法具有固有的随机性。意味着散列可以随着向量发现自身的存储桶而改变地图调整。

image ©
(Reformer:
The Efficient
Transformer)

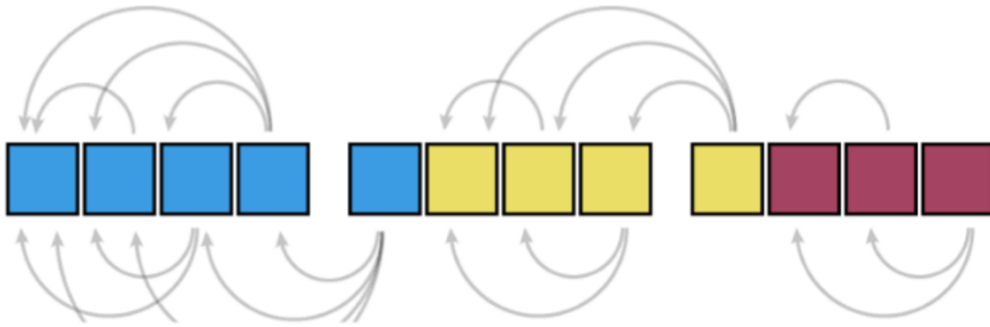
LSH Attention



然后让每个块自己参与。

- 当堂测验 2

How does LSH Attention take advantage of parallel computing?

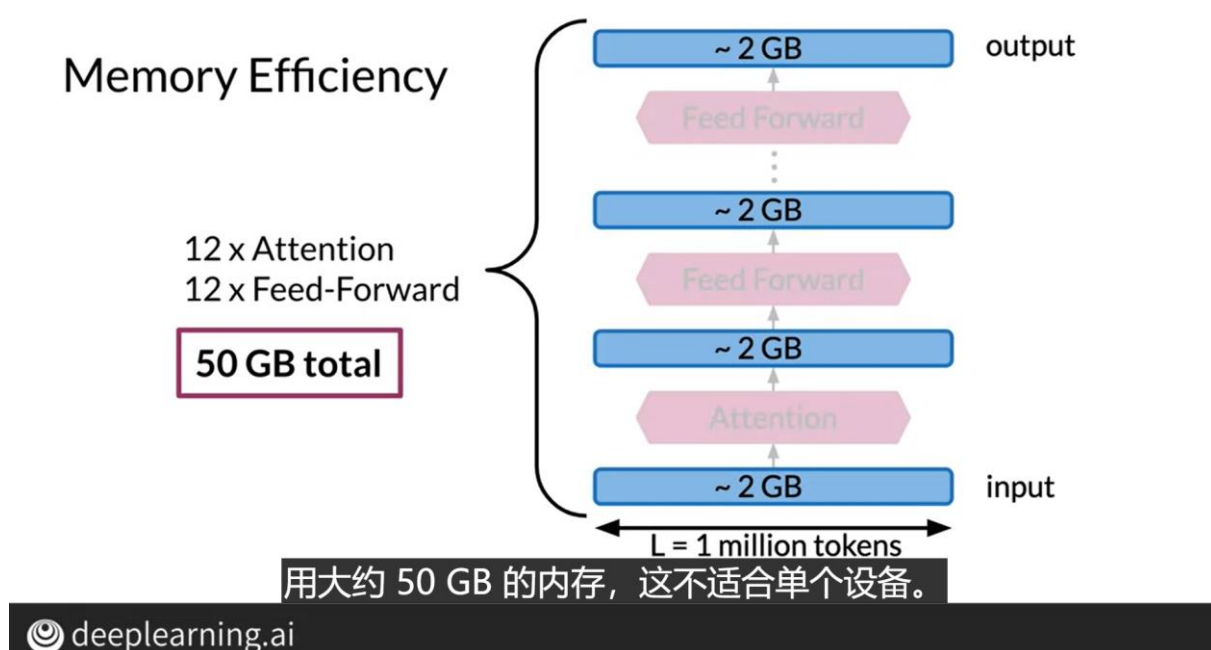


- ☐ By splitting the input sequence.
- ☒ By chunking the LSH buckets.

✓ 正确

5.4 可逆层的设计初衷：更高效的存储

例如，如果您想在这本书上使用变压器，您可能需要输入 100 万个令牌。这些标记中的每一个都会有一个大小相关的特征向量，例如 512。这意味着仅模型的输入就已经有 2GB 的大小。在 16 GB 的 GPU 上，这是总内存预算的八分之一，甚至还没有触及到这些层。说到层，变压器有两种类型的层，注意层是前馈层。一个模型可能有 12 种每种类型的图层。例如，床空间就是这种情况。每层的输入和输出大小也将为 2GB。要通过此模型进行反向传播，前向路径将需要在内存中存储一些中间量。假设您保存的只是来自每一层和第二层之间边界的激活。各个层本身都没有。



您已经看到一个问题，保存这些激活将使用大约 50 GB 的内存，无法在单个设备上容纳。请记住，最新的变压器比 12 层（我在这里使用的）更深，这是变压器面临的第一个基本效率挑战。内存使用量也随着层数线性增长，因此没有办法扩展到百万令牌体系。解决方案请看下一节！

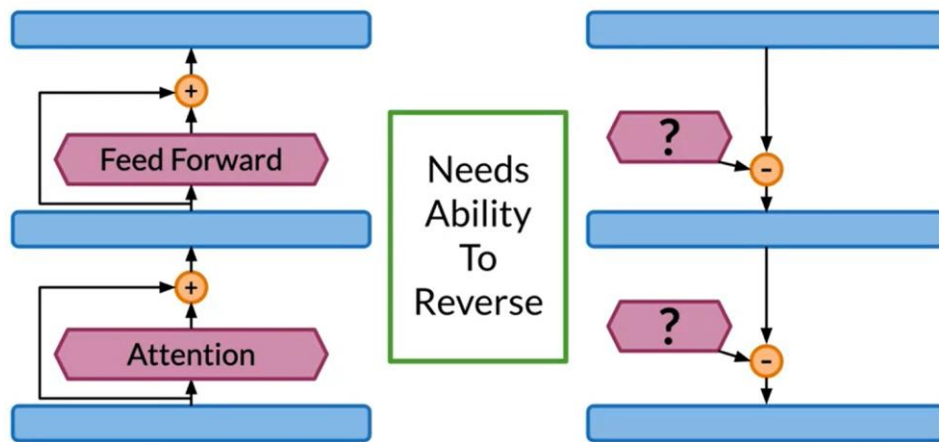
5.5 可逆残留层

在运行大型深度模型时，您经常会耗尽内存，因为每个图层都需要很长时间。我将向您展示如何使用以下方法解决此问题——可逆层。

● 可逆层介绍

这是之前的：

Residual Blocks in Transformer



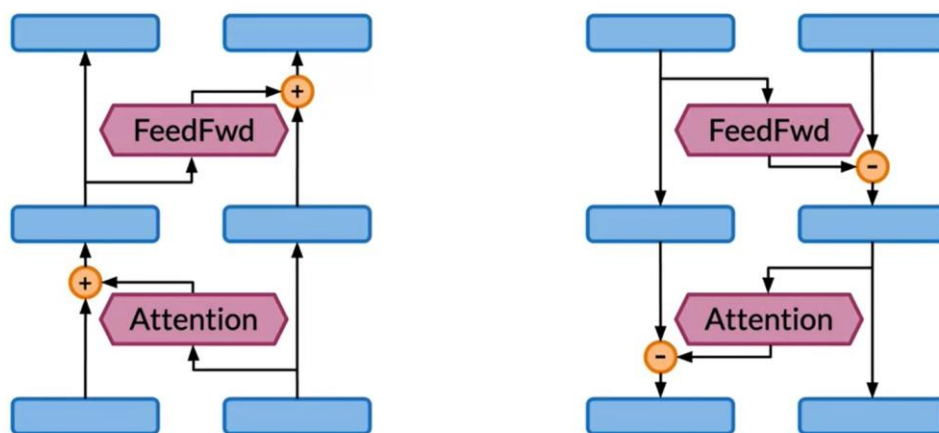
模型的输出开始，以相反的顺序减去残差。

deeplearning.ai

这是之后的可逆层：

分享

Reversible layers



1 层是注意力，第 2 层是前馈。

deeplearning.ai

- 转换方程

Reversible layers equations

Standard Transformer:

$$y_a = x + \text{Attention}(x)$$

$$y_b = y_a + \text{FeedFwd}(y_a)$$

Reversible:

$$y_1 = x_1 + \text{Attention}(x_2)$$

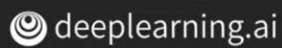
$$y_2 = x_2 + \text{FeedFwd}(y_1)$$

Recompute x_1, x_2 from y_1, y_2 :

$$x_1 = y_1 - \text{Attention}(x_2)$$

$$x_2 = y_2 - \text{FeedFwd}(y_1)$$

您可以重建输入 x_1 和 x_2 , 如下所示,



- 当堂测验

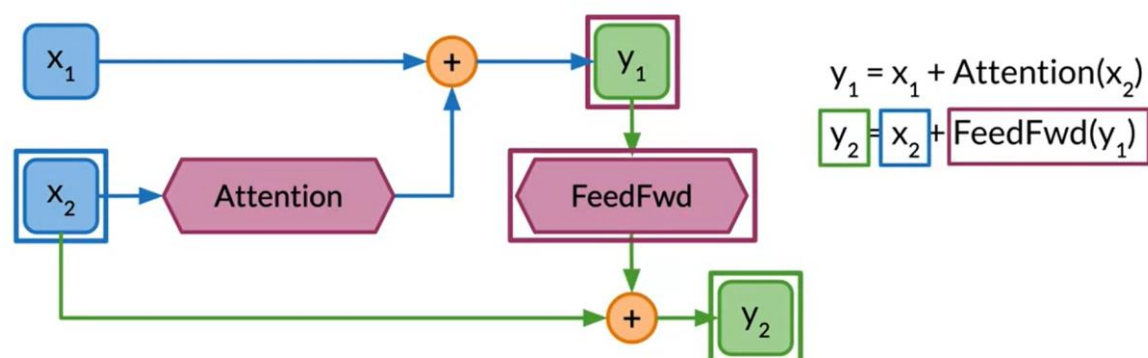
How do reversible residual connections allow the model to save memory?

By not saving the residuals at each step.

- 公式解析

正向计算可逆层公式

Reversible layers equations

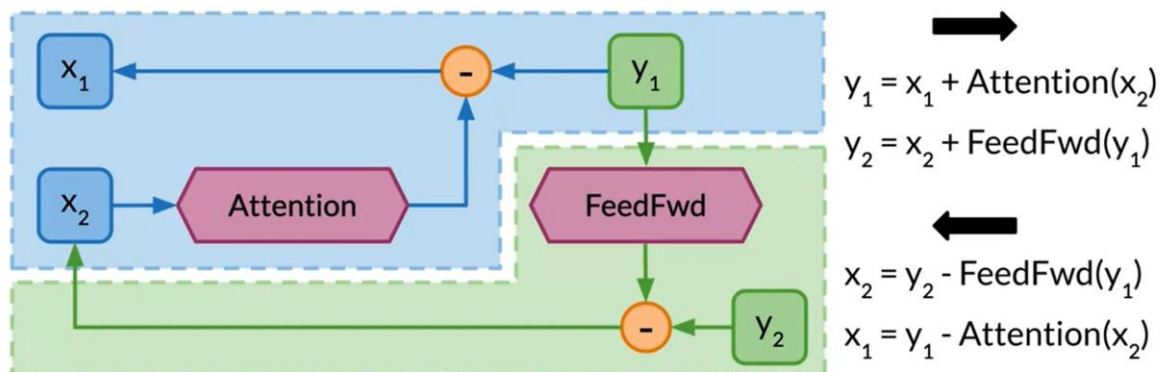


第一步的 y_1 的前向，就是这样。

deeplearning.ai

反向计算可逆层公式

Reversible layers equations

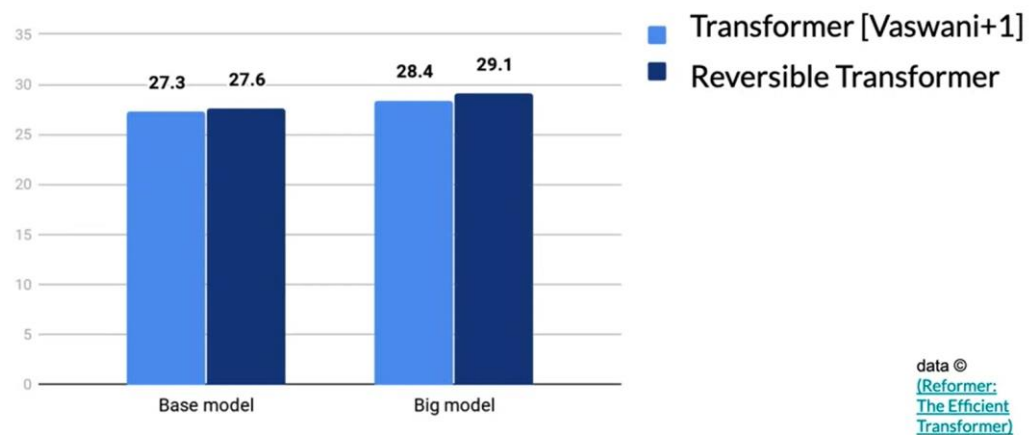


指示信息正在向后流动。

deeplearning.ai

- Transformer 和 可逆变压器的 BLEU 对比

Reversible Transformer: BLEU Scores



事实上，可逆性是一种非常普遍的技术，

5.6 reformer

上一节主要解决了长序列下传统变压器复杂性和内存溢出的问题。本节我们将所有的解决方案放在一起创建一个可逆的变压器模型，称之为改革者。

改革者被设计用于处理最多一百万个单词的上下文窗口，它结合了两项技术来解决 Attention 的关键问题，并用内存分配来限制变压器，应用到长上下文窗口。改革者使用位置敏感的哈希降低复杂度，它还使用可逆残留层以更有效地使用可用内存。

● 模型架构

Reformer

- LSH Attention
- Reversible Layers

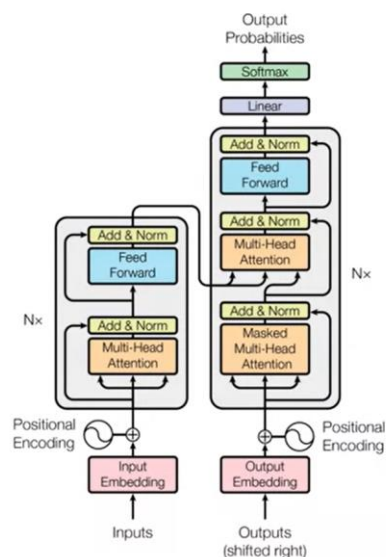


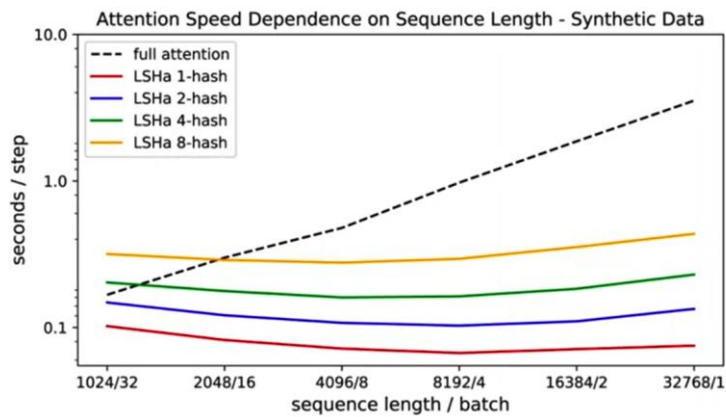
image ©
(Attention Is
All You Need)

以更有效地利用可用的内存。

deeplearning.ai

- Attention 和 LSH 的对比（序列长度和时间）

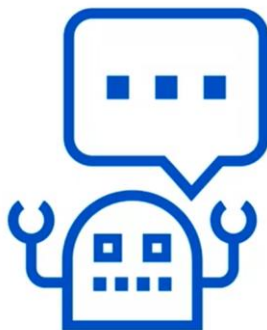
Reformer



deeplearning.ai

- 对话机器人搭建

Chatbot



- Reformer
- MulitiWOZ dataset
- Trax

MultiWoz 是一个非常大的人类对话数据集,

deeplearning.ai

6. 附录：（参考资料）

This course drew from the following resources:

- [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#) (Raffel et al, 2019)
- [Reformer: The Efficient Transformer](#) (Kitaev et al, 2020)
- [Attention Is All You Need](#) (Vaswani et al, 2017)
- [Deep contextualized word representations](#) (Peters et al, 2018)
- [The Illustrated Transformer](#) (Alammar, 2018)
- [The Illustrated GPT-2 \(Visualizing Transformer Language Models\)](#) (Alammar, 2019)
- [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#) (Devlin et al, 2018)
- [How GPT3 Works - Visualizations and Animations](#) (Alammar, 2020)